Contents

Volume 01 — Intro, CNN, Colab labs (L00–L09) **1**
L00: Course introduction 1
L01: Introduction to Generative AI 2
L02: Activation Functions & Loss Functions in Deep Learning 3
L03: Optimizers — Part A 4
L04: Optimizers — Part B 5
L05: Convolutional Neural Network — Part A 5
L06: Convolutional Neural Network — Part B 6
L07: Introduction to Google Colab 7
L08: Practical exercise 1 — CNN on CIFAR-10 8
L09: Practical exercise 2 — Ensemble models 9

Volume 01 — Intro, CNN, Colab labs (L00–L09)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L00: Course introduction

Learning focus

- Place the 12-week course in the sequence after Foundations of Deep Learning.
- List the week-by-week theory map and the matching hands-on track.

Exam lines

- The course is theory and weekly coding, from CNN recap through RAG.
- The video order is the authority for these notes: autoencoders → VAEs → GANs → diffusion → sequence models → transformers/LLMs → RAG/ethics.
- Several topics advertised for weeks 11–12 (RAG, multimodal, ethics, hallucinations) appear in this intro; they are expanded only when a later lecture actually teaches them.

Concept tables

Weeks	Theory focus
1	Deep-learning foundations needed for generative models
2	Autoencoders and representation learning
3–4	Variational autoencoders, advanced VAEs, applications
5–6	GANs, important variants, applications
7–8	Diffusion models
9	Sequence models (RNN/LSTM) and NLP foundations
10	LLM foundations and prompt engineering

Weeks	Theory focus
11	Retrieval-augmented generation (RAG)
12	LLM capabilities, multimodal AI, ethics
Weeks	Labs announced in the intro
——:	———————————–
1–2	CNNs, transfer learning, ensemble models, then the **autoencoder** as the first generative model
3–4	Representation learning: train a **VAE** and a **conditional VAE**
5–6	Adversarial models: **DCGAN**, **cGAN**, **CycleGAN**
7–8	Diffusion: simulate **forward** noise, **reverse** denoising, and a **U-Net**
9–10	Sequence/LLM stack: **RNN**, **LSTM**, **transformer**

L01: Introduction to Generative AI

Learning focus

- Contrast discriminative models with generative models using the lecture's $P(y \mid x)$ vs $P(x)$ / $P(x, y)$ language.
- Explain why **data augmentation** and **oversampling** are not generative modeling.
- Match output type → assumed distribution → activation and loss used in code.
- Locate where a **Gaussian** shows up in VAEs, diffusion, and GANs (preview only).

Exam lines

- MLP / CNN / RNN-style predictors are discriminative: they learn $P(y \mid x)$ and **not** how x was generated.
- Generative modeling learns p_{data} so it can **sample** new realistic x (and optionally x given y).
- Augmentation and oversampling reuse existing points; they are not distribution learning.
- In code, Gaussian / Bernoulli / categorical show up as **linear+MSE**, **sigmoid+BCE**, **softmax+CCE**.
- The same Gaussian idea will reappear in VAE latents, diffusion noise, and GAN z.

Concept tables

Model	Data it fits	Typical tasks
MLP	Tabular (CSV / spreadsheet features)	Classification or regression
CNN	Images	Classification, segmentation, object detection

Model	Data it fits	Typical tasks
RNN / LSTM / GRU	Sequences / time series	"What comes next" (e.g. rainfall tomorrow from 10 days of history)
View	What is learned	When you use it
——	———————–	———————–
Unsupervised generation	$P(x)$ or $p_\theta(x) \approx p_{\text{data}}(x)$	Features only (no labels)
Joint modeling	$P(x, y)$	Features **and** labels
Conditional generation	$P(x \mid y)$ (lecture language: generate x given a chosen class y)	"Only digit 7", "only smiling faces", extra **malignant** medical images

---

L02: Activation Functions & Loss Functions in Deep Learning

Learning focus

- State why a neuron needs a nonlinear activation, and where activations sit (hidden vs output).
- Compute ReLU / Leaky ReLU / ELU on a weighted sum, and pick sigmoid, tanh, or softmax at the output.
- Map this course's models (CNN, autoencoder, VAE, GAN, diffusion, LLM) to typical hidden and output activations.
- Distinguish reconstruction / classification losses from generative-model losses (KL, GAN, noise-prediction).

Exam lines

- No activation ⇒ linear neuron; ReLU-family activations also **gate** negative features.
- Hidden menu: ReLU (dying neurons) → Leaky ReLU (0.01) / PReLU (learned slope) → ELU (smooth exponential) → GELU (transformers) / SiLU (diffusion).
- Output menu: sigmoid $\{0, 1\}$, tanh $\{-1, +1\}$, softmax (multiclass / next token).
- Generative reconstruction uses MSE or BCE on x **vs** $\hat{x}$, not on class labels; LLMs use (sparse) categorical CE; VAEs add KL; GANs and diffusion have their own losses.

Concept tables

If there is no activation	What goes wrong
The weighted sum is emitted unchanged	The neuron is a **linear** model
Stacking linear layers stays linear	The net cannot learn **complex patterns**
No ReLU-family gate	Negative / unused features are not suppressed
Weighted sum x	$\max(0, x)$

If there is no activation	What goes wrong
———————	————
negative (e.g. -3)	0
0	0
positive (e.g. 4)	4

L03: Optimizers — Part A

Learning focus

- Place the optimizer in the train loop: forward pass → loss → gradients → weight update.
- Apply $W_{\text{new}} = W_{\text{old}} - \eta\, \partial L/\partial W$ and explain the minus sign.
- Contrast **SGD**, **mini-batch SGD**, and **batch gradient descent** by when weights are updated.
- Write SGD **with momentum** as an exponentially weighted moving average of gradients.

Exam lines

- Optimizer = use $\partial L/\partial W$ to raise or lower W so loss falls; the minus sign is "move downhill."
- SGD / mini-batch / batch GD differ only in **how many samples** feed one update.
- SGD zigzags because each update sees only the current g_t.
- Momentum replaces g_t by an EWMA v_t ($\beta \approx 0.9$): same-direction gradients **speed up**; a sudden sign flip **damps** rather than snaps.

Concept tables

Gradient $\partial L/\partial W$	Arithmetic	W_{new}	Effect
-2 (negative)	$5 - 0.1(-2) = 5.2$	5.2	**weight increases**
positive	5 minus a positive quantity	< 5	**weight decreases**

Optimizer	Samples per update	Typical path to the minimum
———–	————–	———————–
SGD	1	noisy, much zigzag
Mini-batch SGD	k (hyperparameter)	smoother than SGD
Batch GD	all n	one step per full pass

L04: Optimizers — Part B

Learning focus

- Separate **gradient sign** (direction) from **gradient magnitude** and **learning rate** (step size).
- Explain why a **single** η for millions of weights is a bad idea.
- Write **AdaGrad**'s per-parameter rate from the sum of **squared** past gradients, and state why the rate can die.
- Show how **RMSProp** replaces that sum by an EWMA of g_t^2, and how **Adam** = momentum + RMSProp.

Exam lines

- Sign of g = direction; $|g|$ and η = step size. A huge η with a huge g can slam W (e.g. $5 \to 0$).
- Weights see different $|g|$; one global η is the SGD assumption we drop.
- AdaGrad: $\eta_t = \eta/\sqrt{\sum g^2 + \varepsilon}$ — good adaptivity, **vanishing rate** as the sum grows.
- RMSProp: same denominator idea with **EWMA of** g^2 so old squares fade.
- Adam: **momentum (**β_1**) + RMSProp (**β_2**)**.

Concept tables

Situation	What happens to W
Large $\|g\|$	**Large** change (lecture: $5 \to 4$)
Small $\|g\|$	**Tiny** change (lecture: $5 \to 4.95$)
η	Qualitative result
——–	——————–
small	small change in W
moderate	e.g. $5 \to 4$
high	**abrupt** jump — lecture's scare case: $5 \to 0$

L05: Convolutional Neural Network — Part A

Learning focus

- Describe how a machine stores an image (pixels, height × width, channels).
- List why a fully connected MLP is the wrong first model for images.
- Name CNN applications (classification, detection, segmentation variants).
- Run one **convolution**: kernel, element-wise multiply-and-sum, stride, padding, output size, Keras `Conv2D` arguments.

Exam lines

- Images are $H \times W \times C$ pixel grids; flattening them for an MLP drops neighbourhoods and explodes parameters.
- CNN applications: classify, detect, semantically / instance-segment (sheep, CT tumour, ...).
- A kernel slides, multiply-and-sums, writes a feature map; **many kernels ⇒ many maps**; kernel weights are learned.
- Size: $\lfloor (n + 2p - f)/s \rfloor + 1$, floor if needed; **padding** protects borders; **stride** is how far the kernel jumps.
- Keras: `Conv2D`, `padding="same"|"valid"`, default stride 1.

Concept tables

Image	Channels	Flattened length
28×28 colour	3 (RGB)	$28 \times 28 \times 3 = 2352$
200×200 colour	3	120,000

Task	Lecture example
——	——————–
Image classification	sheep or not; **benign vs malignant** / cancerous vs not
Object detection	several objects; **where** each one is
Image segmentation	keep only the **region of interest**
Classification + **localization**	same class (sheep) **and** a box
Semantic segmentation	road / grass / sheep — one colour per **class**
Instance segmentation	sheep 1, sheep 2, sheep 3 — same class, **different instances**

L06: Convolutional Neural Network — Part B

Learning focus

- Apply **ReLU** to a feature map and explain why it does **not** downsample.
- Count **learnable conv parameters** and the **output tensor size** after conv.
- Run **max / average / min / max-average-min** pooling, including Keras `MaxPooling2D`.
- Stack conv–ReLU–pool, **flatten**, and a **fully connected** classifier (sigmoid vs softmax).

Exam lines

- ReLU after conv zeros negatives and **keeps spatial size**; pooling is the downsampler.
- Conv parameters: $(f_H f_W C_{\text{in}} + 1) \times n_{\text{filters}}$; output spatial size $\lfloor (I - F + 2P)/S \rfloor + 1$ with depth = number of filters.
- Pooling: 2×2 window, default stride 2; max (salient), average, min, or max-average-min; Keras `MaxPooling2D`.
- Stack blocks until the map is small enough; flatten; dense head with C outputs (sigmoid vs softmax).

Concept tables

$f_H \times f_W$	C_{in}	Filters	Arithmetic	Parameters
5×5	3	6	$(25 \times 3 + 1) \times 6$	456 (stated)
3×3	3	64	$(9 \times 3 + 1) \times 64$	1792
5×5	64	128	$(25 \times 64 + 1) \times 128$	204,928

Task	Output neurons	Activation
——	————--	————
Binary	1	**sigmoid**
3-class	3	**softmax**

L07: Introduction to Google Colab

Learning focus

- Open Colab, create a notebook, connect a runtime, and pick CPU / GPU / TPU.
- Run **code** vs **text** cells (`Shift+Enter`).
- Pull a dataset from **Kaggle**, **GitHub**, or **UCI**, land it in **Google Drive**, and paste the path into pandas / folder variables.

Exam lines

- Colab = free cloud Jupyter; connect, then set **Python 3** and **CPU/GPU/TPU** (GPU for long jobs, TPU for huge / NLP).
- Data path: Kaggle / GitHub / UCI → disk → **Google Drive** → mount → **Copy path** into `read_csv` or train/test variables.
- Kaggle also runs notebooks; GitHub also hosts full CNN projects (confusion matrices included).

Concept tables

Field	Options the lecture shows	What to pick
Runtime type	**Python 3**, **R**, **Julia**	Python 3 for this course; R if you are writing R
Hardware accelerator	**CPU**, **GPU**, **TPU**	see below
Runtime version	several	**latest**
Accelerator	When	
——————-	——	
CPU (default)	fine for small demos; lecture selects CPU “for the time being”	
GPU (graphics processing unit)	same code that might take **10–12 hours** on CPU can take **1–2 hours** on GPU	
TPU (tensor processing unit)	**very large** datasets or **NLP**	

L08: Practical exercise 1 — CNN on CIFAR-10

Learning focus

- Load **CIFAR-10**, normalize pixels to $[0, 1]$, and plot a 2×5 sample grid.
- Build a **three-block** Keras CNN (32 / 64 / 128 filters), flatten, dense, **dropout** 0.5, softmax-10.
- Compile with **Adam** + **sparse categorical cross-entropy** + **accuracy**; train **10 epochs**, batch **64**, `validation_split=0.2`.
- Read accuracy/loss curves, test metrics, per-image predictions, and the 10×10 **confusion matrix**.

Exam lines

- CIFAR-10: $50\text{k} + 10\text{k}$ RGB 32×32 images, 10 named classes; pixels $/255$.
- CNN: Conv32–pool, Conv64–pool, Conv128, flatten, Dense128 ReLU, **Dropout** 0.5, Dense10 softmax.
- Adam + **sparse** CCE (integer labels) + accuracy; 10 epochs, batch 64, 20% val.
- This run: train $\sim 75\%$, val $\sim 72\%$, test $\sim 71\%$ / loss 0.82; CM diagonal (e.g. 815, 839) vs off-diagonal errors (35, 16, ...).

Concept tables

Import	Why
`tensorflow as tf`	DL framework
`tensorflow.keras`: **`datasets`**, **`layers`**, **`models`**	CIFAR loader, layer types, Sequential
`matplotlib.pyplot as plt`	plots
`numpy as np`	arrays
`sklearn.metrics`: **`confusion_matrix`**, **`confusion_matrix_display`**	true vs predicted counts
Split	Contents
——-	——-
`x_train`	training **images**
`y_train`	training **labels**
`x_test`	test images
`y_test`	test labels (ground truth to compare with $\hat{y}$)

L09: Practical exercise 2 — Ensemble models

Learning focus

- Contrast **serial** vs **parallel** ensembling; implement **three CNNs in parallel** and **average** their probability vectors.
- Differentiate the three members: plain CNN, CNN + **batch normalization**, CNN + **dropout** 0.5.
- Train each **5 epochs** (batch 64, `validation_split=0.2`, Adam, sparse CCE) and compare **individual val acc** to **ensemble test acc** $\approx 71\%$.

Exam lines

- Parallel ensemble: three CIFAR CNNs, **average softmax vectors**, then argmax.
- Members: (1) two-block CNN, (2) same idea + **batch norm** ($\frac{x-\mu}{\sigma}$), (3) CNN + 50% **dropout**.
- Train **5** epochs, batch **64**, val 20%, Adam, **sparse** CCE, accuracy.
- This run: val $67\%/68\%/61\%$; ensemble test 71.14% — fewer mistakes than any one model.

Concept tables

Import	Role
`tensorflow as tf`	build nets
Keras `datasets`, `layers`, `models`	CIFAR-10, layers, Sequential
`matplotlib.pyplot as plt`	plots
`numpy as np`	arrays
sklearn `confusion_matrix`, `confusion_matrix_display`	true vs predicted

Import	Role
later: `accuracy_score` from sklearn	ensemble accuracy

Split	Samples
Train	50,000
Test	10,000

Contents

Volume 02 — Autoencoders (L10–L20) **1**
L10: Introduction to Autoencoder 1
L11: Reconstruction Loss (MSE, Binary Cross-Entropy) 2
L12: Types of Autoencoders 3
L13: Denoising Autoencoder 4
L14: Sparse Autoencoder 5
L15: Contractive Autoencoder 6
L16: Numerical Example and Limitations of Autoencoders 7
L17: Practical Exercise 1 — Autoencoders 8
L18: Practical exercise 2 — regularizing an autoencoder 9
L19: Intuition behind KL divergence — Part A 10
L20: Intuition behind KL divergence — Part B 11

Volume 02 — Autoencoders (L10–L20)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L10: Introduction to Autoencoder

Learning focus

- Place autoencoders in week 2 as **representation learning**, after week 1's activations, losses, optimizers, and CNNs.
- Contrast **supervised** $x \mapsto y$ mapping with **unsupervised** learning from unlabeled x only.
- Name the three AE blocks—**encoder**, **latent code**, **decoder**—and the two objectives: representation learning and reconstruction.
- Walk a fully connected encoder/decoder on tabular data: $100 \to 75 \to 25 \to 75 \to 100$.
- Distinguish **undercomplete** vs **overcomplete** latent spaces, including the identity-mapping failure mode.

Exam lines

- Autoencoders are unsupervised **representation learners** that reconstruct their own x; they are not $x \mapsto y$ classifiers.
- Encoder compresses x to latent h; decoder expands h to $\hat{x}$. Compression is **not** deletion of features.
- Two objectives: **representation learning** (encoder) and **reconstruction learning** (decoder).
- Tabular example: $100 \to 75 \to 25 \to 75 \to 100$, fully connected, sum + activation at each neuron. Latent size is a hyperparameter.
- **Undercomplete** ($\dim(h) < \dim(x)$) forces useful features. **Overcomplete** ($\dim(h) \geq \dim(x)$) risks copying x and padding zeros.

Concept tables

Block	Role
Encoder	Learns the most important features of x. Output is a **smaller internal representation**.
Latent code h	That compact representation (also: encoded data, latent representation, hidden representation).
Decoder	Maps h back toward the original input. Output is $\hat{x}$ (not called x, because it is reconstructed).
Objective	Who does it
———–	————-
Representation learning	Encoder
Reconstruction learning	Decoder

L11: Reconstruction Loss (MSE, Binary Cross-Entropy)

Learning focus

- Write the encoder/decoder maps $h = G(xW_E)$ and $\hat{x} = F(hW_D)$ with shapes for a 1000×6 example.
- Contrast **generic** weights (W_E and W_D both learned) with the **tied-weight** assumption $W_D = W^\top$.
- Choose **output activation** from the feature type: sigmoid for binary, linear for continuous.
- Derive **MSE** reconstruction loss for real-valued x and **binary cross-entropy** for $0/1$ features, including why BCE has a leading minus.
- Add a **weight regularizer** so W stays small; after training, $W^\star$ yields a good $h^\star$ and an $\hat{x}$ close to x.

Exam lines

- $h = G(xW_E)$, $\hat{x} = F(hW_D)$; for $1000 \times 6 \to 1000 \times 3$, W_E is 6×3 and W_D is 3×6.
- **Generic:** learn both matrices. **Tied:** learn W, decode with $W^\top$.
- Binary x: sigmoid out, **BCE**. Continuous x: linear out, **MSE**. Hidden: nonlinear (ReLU / sigmoid / tanh).
- Autoencoders have no class labels; reconstruction loss compares $\hat{x}$ to x.
- Regularize W (example $\lambda = 0.01$) so weights stay small; $W^\star$ gives a useful $h^\star$ and $\hat{x} \approx x$.

Concept tables

Object	Shape	Meaning
Input X	1000×6	1000 samples, 6 features
Latent H	1000×3	same 1000 samples, 3-D code (undercomplete)
Encoder weights W_E	6×3	maps $\mathbb{R}^6 \to \mathbb{R}^3$
Decoder weights W_D	3×6	maps $\mathbb{R}^3 \to \mathbb{R}^6$
Reconstruction $\hat{X}$	1000×6	6 features back
Setting	What is learned	Cost
———	———––	——
Generic	W_E **and** W_D	More parameters, slower training
Tied weights	Learn only W $(= W_E)$; set $W_D = W^\top$	Same shapes as needed for decode; **one** matrix to train

---

L12: Types of Autoencoders

Learning focus

- Build a **CNN autoencoder** for images: conv + max-pool encoder, upsample-then-conv decoder, ending in sigmoid for binary pixels.
- Compute the lecture's **28×28×1** Keras stack down to a **7×7×256** latent code and back.
- Contrast **nearest-neighbor / bed-of-nails** upsampling with **transposed convolution**, including the **checkerboard** artifact and why AE decoders prefer upsample + convolution.
- Code a **shallow** AE ($784 \to 32 \to 784$) and a **deep** AE ($784 \to 128 \to 64 \to 32 \to 64 \to 128 \to 784$).
- Explain why flattening a 28×28 image into 784 and using a deep MLP AE **loses spatial structure**.

Exam lines

- Image AE: encoder = conv + **max-pool**; decoder = **upsample then conv**; last layer **sigmoid** if pixels are $0/1$.
- Example stack: $28 \times 28 \times 1 \to \ldots \to$ latent $7 \times 7 \times 256 \to \ldots \to 28 \times 28 \times 1$.
- Nearest-neighbor / bed-of-nails upsample have **no** learned params. Transposed conv learns a kernel but can make **checkerboard** artifacts; prefer upsample + conv.
- Shallow: one hidden layer ($784 \to 32 \to 784$). Deep: stacked bottlenecks ($784 \to 128 \to 64 \to 32 \to \cdots \to 784$).
- Flattening images for an MLP AE loses spatial structure; use a CNN AE.

Concept tables

Stage	Operation	Output shape
Input	image	$28 \times 28 \times 1$
Conv2D	64 kernels, 3×3, ReLU, `padding='same'`	$28 \times 28 \times 64$
MaxPool	window 2×2	$14 \times 14 \times 64$
Conv2D	128 kernels, 3×3, ReLU, `same`	$14 \times 14 \times 128$
MaxPool	2×2	$7 \times 7 \times 128$
Conv2D	256 kernels, `same`	$7 \times 7 \times 256$ **(latent)**
Contribution	Where	Result (as taught)
——————–	——–	———————
G_1	1 at top-left	$1 \cdot [4, 3; 2, 1] \to 4, 3, 2, 1$ in the top-left 2×2
G_2	2 at top-right	$2 \cdot 4 = 8$, $2 \cdot 3 = 6$, $2 \cdot 2 = 4$, $2 \cdot 1 = 2$
G_3, G_4	remaining two inputs × kernel	two more 3×3 maps

L13: Denoising Autoencoder

Learning focus

- Say why AEs need **regularization**, especially in the **overcomplete** identity-mapping case.
- Draw the DAE pipeline: corrupt $x \to \tilde{x}$, encode, decode, compare $\hat{x}$ to the **clean** x (never to $\tilde{x}$).
- Name three corruption recipes: **Gaussian**, **masking** ($q = 0.4$), **salt-and-pepper**, with the lecture's numbers.
- Explain **neighboring-pixel dependence** as the reason a DAE can fill missing pixels (toy counts / means vs real **conv** filters).
- Write the DAE equations and training objective (MSE or BCE on clean x vs $\hat{x}$).

Exam lines

- Regularize AEs to stop identity mapping, especially when $\dim(h)$ is large.
- DAE: encode $\tilde{x}$, reconstruct **clean** x. That gap is the constraint.
- Corrupt with Gaussian (example $0.50 \to 0.51$, ...), masking ($q = 0.4$), or salt-and-pepper ($128 \to 0$, $130 \to 255$).
- Restoration uses **neighbor correlation**; toy fill is majority/mean; real images use **conv** kernels (e.g. 32 filters, 3×3).
- Train MSE or BCE on $(x, \hat{x})$; test: noisy in, clean out.

Concept tables

	x_1	x_2	x_3	x_4
Clean x	0.50	0.80	0.20	0.90
After noise $\tilde{x}$	0.51	0.75	0.23	0.82

Corruption	What it does
Gaussian	Add $\mathcal{N}(0, \sigma^2)$
Masking	Each feature $\to 0$ with probability q (lecture $q = 0.4$)
Salt-and-pepper	Random pixels $\to$ extreme intensity (0 or 1 binary; 0 or 255 grayscale)

L14: Sparse Autoencoder

Learning focus

- State the SAE idea: even with a **wide** hidden layer, keep **most units inactive** so the few that fire must carry important features.
- Write the loss as reconstruction **plus** a sparsity penalty controlled by β.
- Define average hidden activation $\hat{\rho}_j$, desired sparsity ρ, and the constraint $\hat{\rho}_j \approx \rho$.
- Use **KL divergence** $\mathrm{KL}(\rho \| \hat{\rho}_j)$ as the penalty, including the active and inactive terms, and the identity case $\rho = \hat{\rho} = 0.05$.
- Follow the lecture's **5-sample / 7-neuron** example: large $\hat{\rho}$ on **H2** and **H6** → large KL → backprop shrinks those units.

Exam lines

- SAE: many hidden units allowed, but **most stay off**; the few that fire must be useful.
- Loss = reconstruction + $\beta\times$ sparsity penalty (KL in this lecture; L1 is the other option).
- $\hat{\rho}_j$ = mean activation of unit j over m samples; target ρ; want $\hat{\rho}_j \approx \rho$.
- $\mathrm{KL}(\rho \| \hat{\rho})$ has an **active** log and an **inactive** log; $\rho = \hat{\rho} = 0.05$ gives $\mathrm{KL} = 0$.
- In the 5×7 example, **H2** and **H6** are over-active; large KL + backprop makes them quieter.

Concept tables

Term	Role
$L_{\text{reconstruction}}$	How well $\hat{x}$ matches x
L_{sparsity}	**Penalty** if too many hidden neurons are active
β	Hyperparameter: **how strongly** to enforce sparsity (small constant)
Term	What it matches
——	——————–
$\rho \log(\rho/\hat{\rho}_j)$	**Active** part: how often the unit **should** fire vs how often it **does**
$(1-\rho)\log\big((1-\rho)/(1-\hat{\rho}_j)\big)$	**Inactive** part: how often it **should** be off vs how often it **is**

L15: Contractive Autoencoder

Learning focus

- State the CAE extra requirement: reconstruct well **and** keep h **stable** when x changes only slightly (noise, brightness).
- Write the **Jacobian** $J_h(x) = \partial h/\partial x$ and the **Frobenius** penalty $\|J_h(x)\|_F^2$.
- Add that penalty to reconstruction loss with weight λ.
- Interpret a large vs small $\partial h_1/\partial x_1$ (sensitivity vs insensitivity).
- Explain the **trade-off**: keep **genuine** variations, suppress **unimportant** ones (noise), using λ and the two loss terms.

Exam lines

- CAE: $\hat{x}$ should match x, and $h(x)$ should match $h(x + \text{small noise})$.
- Penalty is λ times the **Frobenius** norm of $J_h(x) = \partial h/\partial x$.
- Large Jacobian entries = h too sensitive (example $x = (1.5, 2.5, 3.5)$ vs $(1.6, 2.5, 3.4)$).
- L_{rec} vs Jacobian is a trade-off: keep **real** factors, drop **noise**; λ sets the balance.
- Overcomplete identity mapping is the failure mode this regularizer is built to block.

Concept tables

Size of $\partial h_1/\partial x_1$	Meaning	Wanted?
Large	h_1 changes a lot when x_1 changes a little; h_1 is **very sensitive** to x_1	No: reconstruction would **copy noise** into $\hat{x}$

Size of $\partial h_1/\partial x_1$	Meaning	Wanted?
Small	h_1 barely reacts; **insensitive** to small x_1 wiggles	Yes for **unimportant** wiggles
λ	Effect (as taught)	
———–	—————————	
Large	Punish h-movement more → **more stable** features	
Small	Weaker penalty → **more detailed** reconstruction	

L16: Numerical Example and Limitations of Autoencoders

Learning focus

- Run a fully connected AE **forward pass** with the lecture's numbers: $5 \to 4 \to 3 \to 4 \to 5$.
- Use **ReLU** on hidden layers and **linear + MSE** on a continuous output.
- State what training does to weights/biases so $\hat{x}$ approaches x.
- List four **limitations** of AEs (uninterpretable h, weak downstream use, approximation not generation, latent-size trap).
- See why week 3 adds a **VAE** (KL regularizer on the latent space).

Exam lines

- Worked net: $x = (0.8, 0.4, 0.6, 0.2, 0.5)$, $5 \to 4 \to 3 \to 4 \to 5$, ReLU hidden, linear out.
- Spoken checkpoints: $H_1 = (0.72, 0.65, 0.66, 0.77)$, $h = (0.760, 0.714, 0.944)$, $\hat{x}_1 = 0.67838$; then MSE and backprop.
- Low reconstruction error does **not** imply a meaningful or reusable h, and AEs **reconstruct** rather than sample new x.
- Tiny latent loses information; huge latent copies. VAEs add **KL on** h to fix the latent-space issues.

Concept tables

	x_1	x_2	x_3	x_4	x_5
Original x	0.8	0.4	0.6	0.2	0.5
$\hat{x}$ (first coord. spoken)	0.67838	(slide)	(slide)	(slide)	(slide)
x type	Hidden act.	Output act.	Loss		
———	———-	———-	——		
Continuous (this example)	ReLU	**Linear**	**MSE**		
Binary $0/1$	ReLU	**Sigmoid**	**BCE**		

L17: Practical Exercise 1 — Autoencoders

Learning focus

- Load MNIST **without labels**, scale to $[0, 1]$, flatten $28 \times 28 \to 784$.
- Train a **shallow** AE $784 \to 64 \to 784$ (Adam, BCE) and read the parameter counts $50{,}240 + 50{,}960 = 101{,}200$.
- Train a **deep** AE $784 \to 256 \to 128 \to 64 \to 128 \to 256 \to 784$ and compare validation loss to the shallow net.
- Add **L1** (10^{-4}) for a **sparse** AE and see higher loss / worse reconstructions.
- Build a **DAE**: Gaussian noise with `noise_factor=0.5`, clip to $[0, 1]$, train on noisy inputs vs clean targets.

Exam lines

- Drop MNIST labels; reconstruct `x` from `x` (or from `x_noisy` in the DAE).
- Shallow 64-D bottleneck: **101,200** weights; val. loss down to ~ 0.09 in 5 epochs.
- Deep has **484,944** weights; here it does not beat shallow on val. loss in 5 epochs.
- Sparse L1 (10^{-4}) **raises** loss (~ 0.43) and hurts visual reconstructions in this run.
- DAE: `noise_factor=0.5` Gaussian, clip $[0, 1]$, train `(noisy, clean)`; recovers a clean digit from a noisy one.

Concept tables

Import	Role
`numpy` as `np`	Arrays / numbers
`matplotlib.pyplot` as `plt`	Plots
`tensorflow` as `tf`	Deep-learning framework
`mnist` from `tensorflow.keras.datasets`	Handwritten digits **0–9** (10 classes in the dataset, unused as labels)
`Dense`, `Input`	Hidden/output dense layers; input size
`Model`	Functional API model
Split	Count
——-	——:
Train	60,000
Test	10,000

L18: Practical exercise 2 — regularizing an autoencoder

Learning focus

- Implement five regularization methods on the same MNIST autoencoder: **L1**, **L2**, **dropout**, **batch normalization**, and **early stopping**.
- State what each method does to weights or connections (absolute vs squared penalty, dropped edges, per-batch scaling, patience).
- Train with the lecture's optimizer, loss, epochs, and batch size, and compare **validation losses**.
- Read the lecture's ranking: L2 beats L1; among all five runs, **early stopping** is called the best.

Exam lines

- Same data recipe every time: MNIST, float, $/255$, shape `(-1, 784)`, **Adam + BCE**, reconstruct `x` from `x`, **5 × batch 256**.
- L1 taxes $|w|$; L2 taxes w^2; both at $\lambda = 10^{-4}$ on a **128**-unit bottleneck. L2's last val. loss **0.09** beats L1's **0.43**.
- Dropout **0.3** after 256 and after 128 stops co-adaptation so test reconstructions generalize.
- Batch norm uses $(x - \mu)/\sigma$ and reached **0.08** in this run.
- Early stopping **monitors `val_loss`**, **patience 2**, **restores best weights**; here loss still fell to **0.0722** in five epochs, so the callback never clipped training.

Concept tables

Import	Role in the notebook
`numpy` as `np`	Arrays
`matplotlib.pyplot` as `plt`	Plots (available; this session is mostly loss numbers)

Import	Role in the notebook
`tensorflow`	Neural nets
`tensorflow.keras.datasets.mnist`	Handwritten digits **0–9** (10 classes)
* from TensorFlow / Keras	Pull in layers and model helpers in one go
Keras `models`	Build the autoencoder
Regularizers **L1, L2**; **Dropout**; **BatchNormalization**; **EarlyStopping**	The five techniques

Choice	Value
——–	——-
Input size	784
Output activation	**Sigmoid**
Optimizer	**Adam**
Loss	**Binary cross-entropy**
Fit targets	`(x_train, x_train)`
Epochs	**5**
Batch size	**256**
Validation	`(x_test, x_test)`

L19: Intuition behind KL divergence — Part A

Learning focus

- Say why **KL divergence** exists: Euclidean distance is for **points**; KL is for **probability distributions** $P(x)$ vs $Q(x)$ (discrete or continuous).
- Read the three overlap pictures: identical → KL $= 0$; nearby means → small KL; far apart → large KL.
- Write **forward** $D_{\mathrm{KL}}(P \,\|\, Q)$ and **reverse** $D_{\mathrm{KL}}(Q \,\|\, P)$ for sums and integrals.
- Match **forward = mode covering** and **reverse = mode seeking** on the height (160 / 190 cm) and cats-vs-dogs examples.

Exam lines

- KL replaces Euclidean distance when P and Q are **densities**, discrete or continuous.
- Identical densities → KL **0**; slight mismatch → **small**; far means → **large**.
- **Forward** KL makes Q **cover** P (risk: gap samples). **Reverse** KL makes Q **seek one mode** (risk: drop the other mode).
- In a VAE, P is the unknown data distribution and Q_θ is the learned approximation; KL is how we **score** the mismatch. The algebraic identity is the **next** lecture.

Concept tables

Topic this week	Where
Intuition behind KL	This lecture + Part B
VAE architecture	Encoder, **latent space**, decoder (later videos)
ELBO	Evidence lower bound (later)
VAE loss	**Reconstruction + KL**
Reparameterization trick	Later
Numerical example	End of the week
Case	Overlap
——	———
P and Q **identical** (blue P, red Q stacked)	Complete
Nearby means, **good but not perfect** overlap	Partial
Means far, **almost no overlap**	Little

L20: Intuition behind KL divergence — Part B

Learning focus

- Define **self-information** $I(x) = -\log P(x)$ as surprise of **one** outcome, with the weather numbers in **bits**.
- Define **entropy** $H(P)$ as the **average** surprise of a whole distribution (one number for uncertainty).
- Define **cross-entropy** $H(P, Q)$ as surprise of the **true** outcome under the **model's** Q.
- Derive **forward KL** as the lecture did: $D_{\mathrm{KL}}(P \,\|\, Q) = H(P, Q) - H(P)$, down to $\sum P \log(P/Q)$.

Exam lines

- $I(x) = -\log P(x)$: surprise of **one** outcome (sunny **0.193 bits**, rain **3 bits** at these P).
- $H(P)$: average surprise of **all** outcomes (**0.544** here; **1** if 50–50; **0** if deterministic).
- $H(P, Q)$: score Q against true P (small if Q is high on the correct class).
- Forward KL is **cross-entropy minus entropy**, which algebraically is $\sum P \log(P/Q)$.

Concept tables

Outcome x	$P(x)$	Spoken percent
Sunny	**0.875**	87.5% (also said "around 87% / 90%")
Rainy	**0.125**	12.5%
$P(x)$	$I(x)$	Reading
——–	——–	———
Large (sunny happened, as predicted)	**Small**	Little surprise
Small (rain happened)	**Large**	Much surprise

Contents

Volume 03 — VAE family (L21–L30) **1**
L21: Introduction to VAE and the working of the encoder 1
L22: Probabilistic decoder, ELBO, and the VAE loss 2
L23: Reparameterization trick 3
L24: VAE numerical example 4
L25: Practical exercise 1 — variational autoencoder 5
L26: Entanglement, disentanglement, and β-VAE 6
L27: Conditional VAE 7
L28: Latent-space interpolation 7
L29: Practical exercise 2 — Conditional VAE 8
L30: Practical exercise 3 — β-VAE 9

Volume 03 — VAE family (L21–L30)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L21: Introduction to VAE and the working of the encoder

Learning focus

- Contrast autoencoders (**reconstruction**) with VAEs (**generation** of new samples).
- State the generative goal: from finite unlabeled samples, learn $p_\theta(x) \approx p_{\text{data}}(x)$.
- Explain why the encoder learns an **approximate posterior** $q_\phi(z \mid x)$, not the true $p(z \mid x)$.
- List the **Gaussian** assumptions on $p(z)$ and $q_\phi(z \mid x)$, and what the encoder actually outputs (**mean** and **log-variance**).
- Walk the lecture's **latent-size-5** sampling picture.

Exam lines

- AE reconstructs; VAE must **generate** by learning $p_\theta(x)$ from unlabeled samples.
- Encoder's target is $p(z \mid x)$, but evidence $p(x) = \int p(x \mid z)p(z)\,dz$ is **intractable**, so we train $q_\phi(z \mid x)$.
- Prior $p(z)$ and approximate posterior q_ϕ are both taken **Gaussian**; encoder emits μ **and** $\log \sigma^2$; **KL** pins q to the prior.
- z is a **vector**; sampling it is a **random pick** from each coordinate's Gaussian, so one x has **infinitely many** latents.

Concept tables

Object	Distribution	Lives in
Data $x_1, \dots, x_n$	$p_{\text{data}}(x)$	**Data space** (input dimension / structure)
Latent z	$p(z)$	**Latent space** (different dimension / structure)
Autoencoder	VAE	
———--	—–	
Encoder	**Probabilistic encoder**	
Latent code	Latent **distribution**	
Decoder	**Probabilistic decoder**	

L22: Probabilistic decoder, ELBO, and the VAE loss

Learning focus

- Describe the decoder as a net that maps sampled z to **likelihood parameters** (Gaussian vs Bernoulli, by data type).
- Use the lecture's **5-feature / latent-size-2** example to see **infinitely many** z for one x, and slightly different $\hat{x}$ that stay near x.
- Explain why $p(x) = \int p(x \mid z)p(z)\,dz$ is **intractable**, so the encoder never learns the **true** posterior.
- Write **ELBO** as reconstruction minus KL, and the **VAE loss** as **minimize** $-$ELBO.
- State the **closed-form Gaussian KL** and why **naïve sampling of** z **blocks backprop** (reparameterization is the next video).

Exam lines

- Decoder outputs **likelihood parameters**: Gaussian (μ, σ^2) for continuous x, Bernoulli probabilities for binary x.
- One $x \mapsto$ **infinitely many** $z \mapsto$ many nearby $\hat{x}$ (**generation**).
- $p(x) = \int p(x \mid z)p(z)\,dz$ sums **unbounded** contributions; true posterior is **intractable**; encoder uses q_ϕ.
- Maximize **ELBO** $= \mathbb{E}[\log p(x \mid z)] - \mathrm{KL}(q_\phi \,\|\, p)$. Minimize **loss** $= -$ELBO **=** reconstruction (MSE or BCE) + KL.
- Naïve sampling of z **blocks** backprop → reparameterization next.

Concept tables

Data you reconstruct / generate	Likelihood	Decoder outputs	How you get $\hat{x}$
Continuous / real values	**Gaussian**	**Mean and variance**	Theory: **sample** $\hat{x}$ from that Gaussian. Practice (next numerical lecture): the **mean itself** is $\hat{x}$
Binary (each feature / pixel 0 or 1)	**Bernoulli**	A **probability per feature**	$\hat{x}$ sampled from those Bernoullis

Sample	(z_1, z_2)
$z^{(1)}$	$(0.56, -0.11)$
$z^{(2)}$	$(0.43, -0.32)$
$z^{(3)}$	$(0.61, -0.24)$

L23: Reparameterization trick

Learning focus

- Restate the VAE loss (reconstruction + closed-form KL) and why **sampling** z blocks **backprop**.
- Show that **decoder** parameters are fine under ordinary chain rule; the break is $\partial z/\partial \mu$ and $\partial z/\partial \sigma$.
- Write the **reparameterization** $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, and treat ε as a **constant** during backprop.
- Compute $\partial z/\partial \mu = 1$ and $\partial z/\partial \sigma = \varepsilon$, then push gradients into the **encoder weights**.
- Draw the **full VAE architecture** the course withheld until this trick.

Exam lines

- Reconstruction + KL is not enough: **sampling** z is non-differentiable, so encoder ϕ cannot be updated.
- Decoder W, b and even $\partial L/\partial z$ are ordinary calculus; the hole is $\partial z/\partial \mu$ and $\partial z/\partial \sigma$.
- Reparameterization: $z = \mu + \sigma\varepsilon$ with ε **independent** and treated as **constant**. Then $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$.
- Gradients reach encoder parameters; **end-to-end** VAE training works.

Concept tables

Data	Output activation f
Binary	**Sigmoid**
Continuous	**Linear**

Data	Output activation f
Before reparameterization	After
————————	——–
$z \sim \mathcal{N}(\mu, \sigma^2)$ (raw sample)	$z = \mu + \sigma \odot \varepsilon, \varepsilon \sim \mathcal{N}(0, I)$

L24: VAE numerical example

Learning focus

- Run a **full forward pass** of a tiny MLP VAE on one **continuous tabular** row (four features).
- Compute hidden ReLUs, **μ** and **log-variance** (no activation on those heads), **σ²** and **σ**, then **reparameterized** z.
- Decode with ReLU then **linear** output; compare $\hat{x}$ to x.
- Plug the lecture's numbers into **MSE** reconstruction and **closed-form Gaussian KL**, then add them.

Exam lines

- Tiny MLP VAE on $x = (5.1,\ 3.5,\ 1.4,\ 0.2)$: hidden ReLU **(1.07, 1.93, 0.87)** → **μ=(0.043, 0.551)**, **log σ²=(0.732, 0.302)**, **σ₁=1.442**.
- First $\varepsilon = (0.5, -0.2)$ → $z = (0.764,\ 0.318)$; other ε give other z from the **same** Gaussian.
- Decoder ReLU **(0.6538, 0, 0.0938)** → linear $\hat{x}_1 = 0.10262$; reconstruction is bad because weights are random.
- $L_{\text{recon}} = 9.7111$, KL $= 0.352$, $L = 10.0631$. Training would now update parameters. Week 4 lab implements a real VAE.

Concept tables

Choice	Lecture
Data	**Continuous**, **tabular** (rows × columns)
Features / input neurons	**4**
Spoken coordinates	**5.1**, **3.5**, **1.4**, and a fourth input into weight **0.2**
Consistent fourth value	**0.2** (Iris-style row $(5.1,\ 3.5,\ 1.4,\ 0.2)$ makes the spoken $h_1 = 1.07$)
Encoder hidden	**3** neurons, **ReLU**, bias **0.1**
Latent size	**2** (two μ, two log-σ²)
Decoder hidden	**3** neurons, **ReLU**, bias **0.1**
Decoder output	**4** neurons, **linear** (continuous reconstruction)

Choice	Lecture
Weights	**Random** (can be negative, e.g. -0.7)
Hidden unit	Pre-activation h
———	——————-
1	**1.07**
2	**1.93**
3	**0.87**

L25: Practical exercise 1 — variational autoencoder

Learning focus

- Contrast a **vanilla autoencoder** (fixed latent vector) with a **VAE** ($z = \mu + \sigma\varepsilon$).
- Build the lecture's Keras VAE on **MNIST**: **784 → 256 → 16** latents → **256 → 784**, with a **sampling layer**.
- Code **BCE reconstruction + closed-form KL**, train with **Adam**, **10** epochs, batch **128**.
- Plot original vs reconstructed digits and state the lecture's diagnosis: missing **labels** → blurry / wrong digits, which a **conditional VAE** will fix.

Exam lines

- VAE lab: MNIST **60k / 10k**, $/255$, flatten **784**, **no labels**.
- Encoder **256 ReLU + 16-D** μ and log-σ²; $z = \mu + \exp(0.5 \log \sigma^2) \odot \varepsilon$; decoder **256 ReLU → 784 sigmoid**.
- $L = \text{BCE} + \text{KL}$ with the Gaussian closed-form KL; **Adam**, **10** epochs, batch **128**.
- Final losses ≈ KL 3.4×10^{-5}, recon **0.2643**, total **0.2644**; reconstructions are **784-D** again.
- Plots show **class mix-ups** because y is unused — motivation for **CVAE**.

Concept tables

Autoencoder	VAE in this lab
Encoder x → **one fixed** latent vector	x → **μ and variance** of the pixel distribution
Latent Cannot be resampled	$z = \mu + \sigma\,\varepsilon$ with noise **ε**
Decoder z → reconstruction	Same, but z is **stochastic**

Import	Use
———	—–
`tensorflow as tf`	Deep-learning framework
`keras.layers`	Layers: input, hidden / output layers;
`keras.models`	Model

Autoencoder	VAE in this lab
`matplotlib.pyplot` as `plt`	Original vs reconstruction plots

L26: Entanglement, disentanglement, and β-VAE

Learning focus

- Explain why a fully connected encoder produces an **entangled** z: each coordinate mixes many factors of variation.
- Contrast that with a **disentangled** latent space, where each z_i mainly controls one factor (hair, lighting, smile, …).
- Write the **β-VAE** loss and say what $\beta = 1$ vs $\beta > 1$ does to the KL term.
- Use the lecture's two-dimensional KL examples to connect "small μ" ↔ closer to the prior ↔ one-factor codes.
- List the three practical fixes when there are **more factors than latent dimensions**: larger z, moderate β, **β-annealing**.

Exam lines

- Fully connected μ, σ make each z_i a **mixture** of factors: that is **entanglement**. Editing one coordinate moves several attributes.
- **Disentanglement** means one coordinate ≈ one factor (hair vs smile vs pose on the face grids; thickness vs slant on digits).
- β-VAE is the VAE loss with weight β on the KL. $\beta = 1$ is ordinary VAE; $\beta > 1$ **encourages** (does not certify) disentanglement; β too large **hurts reconstruction**.
- Small μ ⇒ small KL ⇒ codes closer to the prior and less mixed; large μ ⇒ large KL ⇒ mixed, high-capacity z.
- If factors outnumber latent dimensions, raise **latent size**, use a **moderate** β, or **anneal** β from ≈ 0 upward.

Concept tables

Factor	What it means on a digit
Thickness	Thin vs thick stroke
Slant	Tilted vs upright
Rotation	How the glyph is rotated
Position	Where the digit sits in the frame

Coordinate	Should mainly control
z_1	Hair
z_2	Lighting
z_3	Skin tone
z_4	Smile

Factor	What it means on a digit
z_5	Pose

L27: Conditional VAE

Learning focus

- Recap the **unconditional** VAE path $x \to z \to \hat{x}$ and why generation is not class-controlled.
- State the CVAE change: extra condition y into **both** encoder and decoder.
- Explain why z in a CVAE can drop class identity and keep **within-class style**.
- Write the CVAE loss as the VAE loss with y inserted, and follow the lecture's numerical comparison (4 features $\to$ 6 inputs with one-hot y).

Exam lines

- Vanilla VAE: $x \to z \to \hat{x}$. You cannot request a **specific class**.
- CVAE: pass y **into encoder and decoder**. Reparameterization becomes $z = \mu(x, y) + \sigma(x, y) \odot \varepsilon$.
- z no longer has to store class identity, so it can store **within-class style**; generation is **controlled** by y.
- Loss is the VAE pair (reconstruction + KL) with y in both q and p.
- In the reused week-3 example, four features plus a 2-d one-hot become **six** inputs, extra weights, a **richer** hidden code, and a **different** $\hat{x}$.

Concept tables

Network	Standard VAE input	CVAE input
Encoder	x	x **and** y
Reparameterization	$\mu(x), \sigma(x)$	$\mu(x, y), \sigma(x, y)$
Decoder	z	z **and** y
Object	Dimension	
——–	———–	
x	4	
one-hot y	2	
encoder input $[x; y]$	**6**	

L28: Latent-space interpolation

Learning focus

- State why interpolation is done in z, not in pixel space.

- Write the linear blend $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ and evaluate the lecture's $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$.
- Decode each intermediate z to get a **smooth morph** (digit 3 → digit 8).
- Name the medical **missing-scan** use case (early tumor vs late tumor, nothing in between).

Exam lines

- Interpolation for generation happens in **latent space**, then you **decode**. You cannot mix the two photographs directly and call it a VAE interpolation.
- $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ with $\alpha \in [0, 1]$. Endpoints recover the two encodings; $0.25/0.5/0.75$ are the lecture's interior steps.
- Componentwise mix if $\dim z > 1$. Each mixed z has the **same** length as z_A and z_B.
- Visual result: a **systematic** morph (3 → 3-like → mix → 8-like → 8), useful when **intermediate observations are missing** (tumor-progression example).

Concept tables

α	Algebra	Whose code you get
0	$1 \cdot z_A + 0 \cdot z_B$	exactly z_A → decode to image **3**
1	$0 \cdot z_A + 1 \cdot z_B$	exactly z_B → decode to image **8**
in between	a convex combination	a code **on the segment** $z_A \to z_B$
Code you decode	Image you should see	
———	——————-	
$z_A = z(0)$	$\hat{x}_A$ (reconstructed 3)	
$z(0.25)$	3-like intermediate	
$z(0.50)$	mixed 3/8	
$z(0.75)$	8-like intermediate	
$z_B = z(1)$	$\hat{x}_B$ (reconstructed 8)	

L29: Practical exercise 2 — Conditional VAE

Learning focus

- Build a **CVAE** on **MNIST** in TensorFlow/Keras, using the **same** 28×28 digits as the previous VAE lab so the two are comparable.
- One-hot the labels, concatenate y to **encoder and decoder**, keep the VAE sampling layer.
- Train with **BCE reconstruction + KL**, **Adam**, and generate a **class-specific** digit (the demo uses **7**).

Exam lines

- CVAE = VAE sampling + **one-hot** y **concatenated on both sides**.
- MNIST, 784, latent **16**, hidden **256 ReLU**, output **sigmoid 784**, **Adam**, **10** epochs, batch **128/256**.
- Loss is still **BCE + KL**; epoch-10 total 0.2160, smaller than the unconditioned VAE run.
- Decode $(z, y{=}7)$ to emit **that** class on demand.

Concept tables

Import	Role in this notebook
TensorFlow	Deep-learning framework
Keras `layers`, `Model`	Input / hidden / output layers; wrap encoder, decoder, CVAE
NumPy	Arrays; one-hot is numeric
`matplotlib.pyplot` as `plt`	Show the generated digit

Digit	One-hot (length 10)
—:	———————
0	`1 0 0 0 0 0 0 0 0 0`
1	`0 1 0 0 0 0 0 0 0 0`
2	`0 0 1 0 0 …`
9	`0 0 0 0 0 0 0 0 0 1`

L30: Practical exercise 3 — β-VAE

Learning focus

- Implement **β-VAE** on MNIST: same encoder/decoder skeleton as the VAE labs, with a scalar β **multiplying the KL**.
- Sweep $\beta \in \{0.5, 1, 2, 4, 10\}$ and record reconstruction vs KL.
- Read the plot **"effect of beta on beta variational autoencoder"** and the reconstructed **digit 7** as β grows.

Exam lines

- β-VAE loss: RL $+ \beta \cdot$ KL. $\beta = 1$ is vanilla VAE; sweep **0.5, 1, 2, 4, 10**.
- MNIST **60k / 10k × 784**, latent **10**, **256 ReLU**, **sigmoid 784**, **Adam**, **5** epochs, batch **128**, **no** labels.
- Measured trade-off: $\beta \uparrow \Rightarrow$ **RL↑, KL↓**, digit **7** turns to mush by $\beta = 10$.
- Plot title and markers as above; use the trackers so all five runs are comparable.

Concept tables

Split	Samples	Features
Train	**60,000**	784
Test	**10,000**	784
Name	Value in the demo	
——	————————-	
Latent dimension	**10** (784 down to 10 codes). The sampling-layer recap still says “16” once — leftover wording from the VAE lab; the **decoder input and the run** use **10**.	
β grid	**`[0.5, 1, 2, 4, 10]`**	
Epochs	**5** (kept small because **five models** train; you may raise it)	
Batch size	**128**	
Optimizer	**Adam**	
`verbose`	**1** (one-line loss print per epoch)	

———————————————————————

Contents

Volume 04 — GANs (L31–L43) **1**
L31: Motivation for GANs . . . 1
L32: GAN architecture . . . 2
L33: GAN objective and loss functions . . . 2
L34: GAN Convergence and Nash Equilibrium . . . 3
L35: Introduction to DCGAN . . . 4
L36: Practical Exercise 1 — Vanilla GAN . . . 5
L37: Practical Exercise 2 — DCGAN . . . 6
L38: Conditional GAN . . . 7
L39: Pix2Pix GAN . . . 8
L40: CycleGAN . . . 8
L41: StyleGAN . . . 9
L42: StyleGAN 2 . . . 10
L43: Practical exercise 1 — CycleGAN (advanced GAN lab) . . . 11

Volume 04 — GANs (L31–L43)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L31: Motivation for GANs

Learning focus

- List the **six VAE limitations** this lecture uses as the reason to leave explicit likelihoods.
- State the GAN **motivation** in one sentence: realistic samples **without** a prescribed $p(x)$ or reconstruction loss.
- Define **adversarial learning** as two networks that compete: **generator** vs **discriminator**.

Exam lines

- VAEs are limited by **Gaussian posterior/prior**, **pixel MSE averaging** (3 vs 8 with similar z), **rec↔KL trade-off**, **posterior collapse**, **likelihood choice**, and **missing fine texture**.
- GAN motivation: realistic samples **without** writing $p(x)$ or a reconstruction term.
- GAN = two nets: $G(z) \to$ fakes, D scores real vs fake; they **compete** and G trains on D's feedback.

Concept tables

Piece	Assumption
Approximate **posterior** $q(z \mid x)$	**Gaussian**, so the encoder only emits μ and $\log \sigma^2$
Prior $p(z)$	**standard normal** $\mathcal{N}(0, I)$
Data type	Decoder emits
———–	———————
Continuous	means
Binary	per-bit probabilities

L32: GAN architecture

Learning focus

- Trace **generator training**: $z \to G_\phi(z) = \hat{x} \to D_\theta(\hat{x}) = \hat{y}$, BCE with **target** $y = 1$, update **only** ϕ.
- Trace **discriminator training**: real branch ($y = 1$) and fake branch ($y = 0$), $L_D = L_{\text{real}} + L_{\text{fake}}$, update **only** θ.
- Derive both losses from **binary cross-entropy** and write the chain-rule updates.
- Reproduce the lecture's probability examples (0.2 vs 0.9 / 0.8).

Exam lines

- G_ϕ: Gaussian $z \to$ image $\hat{x}$ **matching real shape** $\to D_\theta(\hat{x})$; $L_G = -\log D_\theta(G_\phi(z))$ with $y = 1$; update ϕ, freeze θ.
- D_θ: real x wants score ≈ 1 ($y = 1$); fake $G_\phi(z)$ wants score ≈ 0 ($y = 0$); $L_D = L_{\text{real}} + L_{\text{fake}}$; update θ, freeze ϕ.
- Same BCE, different targets, **alternating** freezes. Next lecture turns this into a **min / max** objective.

Concept tables

Who is training	Input to D	Target y
Generator	fake $\hat{x}$	**1** (fool D)
Discriminator	real x	**1**
Discriminator	fake $\hat{x}$	**0**

L33: GAN objective and loss functions

Learning focus

- Re-derive L_{real}, L_{fake}, and L_D from BCE (same algebra as Lec 32).
- Turn **minimize** L_D into **maximize** a value function V_D, then write **expectations** for minibatches.

- Explain **why** $\max_\theta V$ pushes $D_\theta(x) \to 1$ and $D_\theta(G_\phi(z)) \to 0$, using the lecture's $\log 0.1 / \log 0.5 / \log 0.9$ table.
- Write the **generator** value function (optimize **only the fake term**) and the combined **minimax** GAN objective.

Exam lines

- $L_D = -\log D(x) - \log(1 - D(G(z)))$ on one pair; **min** L_D = **max** V_D with the minus dropped.
- Batches replace single logs by **expectations** over p_{data} and p_z.
- $\max V_D$ raises $\log D(x)$ ($D(x) \to 1$) and $\log(1 - D(G(z)))$ ($D(G(z)) \to 0$); the $0.1/0.5/0.9$ log table is the proof.
- G cannot touch the real term; it **minimizes** $\log(1 - D(G(z)))$, i.e. wants $D(G(z)) \to 1$ (and hits $\log 0$ if it fully succeeds).
- Full GAN: $\min_G \max_D V(G, D)$ — two opposite objectives on the same V.

Concept tables

$D_\theta(x)$	$\log D_\theta(x)$ (approx.)	Reading
0.1 (untrained, poor)	-2.303	D fails on reals
0.5 after some training	-0.693	improving
0.9 later	-0.105	near the ceiling
$D_\theta(G_\phi(z))$	$1 - D$	$\log(1 - D)$
———————:	——:	————:
0.9 (bad D)	0.1	-2.303
0.5	0.5	-0.693
0.1 (good D)	0.9	-0.105

L34: GAN Convergence and Nash Equilibrium

Learning focus

- State what $p_{\text{data}}(x)$ and $p_g(x)$ are, and how generated samples are produced from z.
- Describe the **start of training**: noisy G, a strong D, and $p_g \neq p_{\text{data}}$.
- Track how discriminator feedback moves p_g toward p_{data} and $D(G(z))$ toward 0.5.
- Define **GAN convergence** and **Nash equilibrium** in the lecture's two-player language.

Exam lines

- Real data: $x \sim p_{\text{data}}(x)$. Fakes: $z \sim \mathcal{N}$, then $G_\phi(z)$ with collection $p_g(x)$.
- At init, $p_g \neq p_{\text{data}}$; D scores reals near 1 and fakes near 0.
- Training updates ϕ from D's feedback; p_g moves toward p_{data} and $D(\text{fake})$ climbs toward 0.5.

- **Convergence:** $p_g = p_{\text{data}}$ and $D \equiv 0.5$ on both. Same state is **Nash**: neither player improves by changing only its own weights.

Concept tables

Source	What the images look like at initialization
Real $x \sim p_{\text{data}}$	Actual faces
Fake $G_\phi(z)$	Noise / disturbed / unrealistic blobs
Input	Discriminator output (probability "real")
——-	————————————————-
Real face	closer to 1
Generated blob	closer to 0

L35: Introduction to DCGAN

Learning focus

- Recap vanilla GAN as two networks with **opposite** (minimax) objectives.
- List four **limitations** of GANs taught here: no convergence guarantee, mode collapse, vanishing gradients, hyperparameter sensitivity.
- State the **DCGAN design rules** (strided / transposed conv, batch norm, no FC, activations).
- Sketch the lecture's **64×64 RGB** generator and discriminator tensor sizes.

Exam lines

- Vanilla GAN is a minimax game; it can fail to converge, collapse modes, starve G of gradients, and blow up if you retune mid-run.
- DCGAN: strided conv in D, transposed conv in G, BN both sides, drop FC, ReLU/tanh in G, Leaky ReLU/sigmoid in D.
- Lecture's picture: 100-D $z \to 4 \times 4 \times 1024 \to 64 \times 64 \times 3$; D mirrors that down to $4 \times 4 \times 512$ then sigmoid.
- StyleGAN / BigGAN / WGAN are taught as **add-ons on the DCGAN foundation**, not replacements of that core.

Concept tables

Network	Input	Job
Generator G	latent noise z	Emit images that D will accept as real
Discriminator D	real images **and** G's images	Tell real from generated
Piece	Vanilla GAN	DCGAN

Network	Input	Job
——-	——--	——–
Discriminator spatial ops	pooling	**strided convolution**
Generator spatial ops	(dense upsample)	**transposed convolution** (also called deconvolution / **fractionally strided** convolution)
Batch normalization	not used	**BN in G and D**
Fully connected layers	present	**removed** in deeper architectures
G activations	—	**ReLU** hidden; **tanh** at output
D activations	—	**Leaky ReLU**; **sigmoid** at last layer (real vs fake: 0 or 1)

L36: Practical Exercise 1 — Vanilla GAN

Learning focus

- Implement a **vanilla GAN** in TensorFlow/Keras on **Fashion MNIST**.
- Normalize pixels to $[-1, 1]$ to match **tanh**, flatten $28 \times 28 \to 784$, and drop labels / test split.
- Build MLP **generator** and **discriminator**, a custom `GAN` model class, **Adam + binary cross-entropy**, and plot both losses.
- Read the loss curves and the 4×4 grid of generated garments.

Exam lines

- Unsupervised Fashion MNIST: **60k × 784**, pixels in $[-1, 1]$, no labels/test.
- G: $100 \to 256 \to 512 \to 1024 \to 784$ tanh, Leaky ReLU 0.2; D is the reverse plus sigmoid.
- Train with a Keras `GAN` class, two Adams ($\beta = 0.5$), BCE, optional 0.05 label noise, misleading ones for G.
- Early: D loss $\ll$ G loss; later they track; 16 grayscale samples already look like garments.

Concept tables

Network	Role in this lab
Generator	Noise $z \rightarrow$ **fake** 28×28 fashion image
Discriminator	Classifier: **real** Fashion MNIST vs **fake**

Class	Item
—:	——
0	T-shirt / top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

L37: Practical Exercise 2 — DCGAN

Learning focus

- Explain why **convolutions** beat the Lec 36 MLP: spatial structure, feature hierarchy, fewer parameters.
- Rebuild Fashion MNIST as $28 \times 28 \times 1$, with shuffle/batch pipeline, `Conv2DTranspose` generator and strided-conv discriminator.
- Reproduce BN + `use_bias=False`, padding, stride-2, Leaky ReLU 0.2, dropout 0.3, Adam 2×10^{-4}, $\beta = 0.5$, BCE **`from_logits=True`**.
- Compare sample quality and param counts to vanilla GAN; note the missing **class control** (teaser for cGAN).

Exam lines

- Keep Fashion MNIST as $28 \times 28 \times 1$; shuffle 60k; batch 128; $z \in \mathbb{R}^{100}$.
- G: dense $7 \times 7 \times 256$ (no bias) $\rightarrow$ Conv2DTranspose + BN + ReLU $\rightarrow$ tanh; $2\,330\,944$ params.
- D: strided conv 64 then 128, Leaky ReLU 0.2, dropout 0.3, dense 1; $212\,865$ params.
- Adam 2×10^{-4}, $\beta = 0.5$, BCE with logits; 20 epochs end near $G\,0.81$, $D\,1.32$.
- Better images than vanilla GAN, still **unconditional** — that is the hook for Lec 38.

Concept tables

Vanilla GAN	DCGAN
Fully connected	**Convolutional**
Spatial structure discarded	Spatial structure **preserved** (padding helps keep borders)

Vanilla GAN	DCGAN
No explicit hierarchy	**Low-level** (edges, corners) → **mid-level** (textures, shapes) → **high-level** (object parts)
More parameters	**Fewer** parameters, less memory, faster training
Knob	Value
——	——:
Buffer / training size	**60 000**
Batch	**128**

L38: Conditional GAN

Learning focus

- Restate vanilla GAN as a **minimax / cricket-style** adversarial game, and why z alone cannot request "digit 5."
- Draw the **cGAN** architecture: condition y into **both** G and D.
- Write **real / fake** D **losses**, the **non-saturating** G **loss**, and the **conditioned minimax** $V(D, G)$.
- Contrast vanilla vs conditional GAN and list the lecture's applications (including the Pix2Pix teaser).

Exam lines

- Vanilla z has no semantic map to classes; you cannot request "digit 5."
- cGAN feeds y to **both** G and D; $G(z, y)$ is the controlled fake.
- $D(x, y) \to 1$, $D(G(z, y), y) \to 0$ when training D; G pushes the fake score to 1.
- Practical $L_G = -\mathbb{E}[\log D(G(z, y), y)]$; objective is minimax V **conditioned on** y.
- Same game as vanilla GAN, plus **label-consistent** (or text/image-conditioned) samples.

Concept tables

Sample	$G(z)$ might be
z_1	digit **3**
z_2	digit **7**
z_3	digit **1**
$D(x, y)$	$\log D(x, y)$
——:	——:
0.9	≈ -0.045
0.2	≈ -0.70

L39: Pix2Pix GAN

Learning focus

- Place Pix2Pix as a **paired image-to-image** specialist of **cGAN**.
- Explain why the generator is a **U-Net** (skip connections) and the discriminator a **PatchGAN**, including the 70×70 patch result.
- Write G**'s loss** $L_{\text{cGAN}} + \lambda L_1$ with $\lambda = 100$, and D's real+fake BCE.
- State training details (alternating Adam steps) and that D **is dropped at inference**.

Exam lines

- Paired translation: sketch↔photo, map↔satellite, LR↔HR, inpainting-style repair.
- U-Net skips restore spatial detail the encoder would drop.
- PatchGAN averages $n \times n$ real/fake votes; paper's best patch 70×70.
- $L_G = L_{\text{cGAN}} + 100\, L_1$; L_D = real ones + fake zeros; alternate Adam 2×10^{-4}.
- Inference = generator only.

Concept tables

Input	Output
Edges / **sketch**	**Photo**
Map	**Satellite** image
Sketch	**Painting**
Low resolution	**Super-resolution**
Damaged image	**Repaired** reconstruction
	Conditional GAN
–	———————–
Scope	**Generic** conditional generation
Condition y	labels, **text**, **images**, …
Generator	ordinary net (as in Lec 38)
Discriminator	ordinary net

L40: CycleGAN

Learning focus

- Explain the **paired-data** bottleneck of Pix2Pix and why **unpaired** translation needs CycleGAN.
- Name the **four networks** G, F, D_Y, D_X on domains X (horse) and Y (zebra).
- State **cycle consistency**: horse→zebra→horse returns the **same** horse (and the backward cycle).
- Write the two **adversarial** losses, L_1 **cycle** loss, $\lambda = 10$, and the joint minimax objective.

Exam lines

- Unpaired $X \leftrightarrow Y$: no matched sketch/photo required.
- $G : X \to Y$, $F : Y \to X$, D_Y judges Y, D_X judges X.
- Cycle consistency: reconstruct the **same** horse/zebra, not a random other one.
- $\mathcal{L} = \mathcal{L}_{\text{GAN}}(G, D_Y) + \mathcal{L}_{\text{GAN}}(F, D_X) + 10\,\mathcal{L}_{\text{cyc}}$.
- G, F min; D_X, D_Y max.

Concept tables

Forward	Backward (also learned)
horse → zebra	zebra → horse
winter → summer	summer → winter
photo → painting style	painting → photo
sketch of a bag → photo of a bag	(unpaired collections)
Network	Map / job
———	———–
Generator G	$X \to Y$ (horse → zebra)
Generator F	$Y \to X$ (zebra → horse)
Discriminator D_Y	real **zebra** vs generated zebra $G(x)$
Discriminator D_X	real **horse** vs generated horse $F(y)$

L41: StyleGAN

Learning focus

- Name the **fine-control** failures of a vanilla $G(z)$ (hair color, smile, glasses) and define **latent-space entanglement**.
- Draw StyleGAN's pipeline: $z \xrightarrow{F} w$ (**8-layer MLP**), **affine** style $y = Aw + b$, **synthesis** from a $4 \times 4 \times 512$ constant.
- Write **AdaIN**: normalize feature maps, then restyle with (y_s, y_b).
- Place **18 convolutions**, resolution-dependent attributes, and **noise injection** (scale B).

Exam lines

- Entanglement: one z-coordinate moves many attributes because G copies correlations in p_{data}.
- StyleGAN inserts mapping F ($8\times$ FC, 512-D) so w can be traversed **per style**.
- Images grow from a **constant** $4 \times 4 \times 512$ plus **AdaIN** styles, not from feeding z into the first conv.
- AdaIN: $\hat{x} = (x - \mu)/\sigma$, then $y_s \hat{x} + y_b$.
- 18 convs, upsample every two layers; noise scaled by B for fine stochasticity.

Concept tables

You want	Can you change only that?
Butterfly **wing color**, keep pose	**No**
Face hair **brown → blonde**	No mapped latent coordinate
Remove a smile	No
Add glasses	No
Traditional GAN	StyleGAN
——————–	———-
z goes **straight** into G	z goes through a **mapping network** F first
Image synthesis starts from z	Synthesis starts from a **learned constant** tensor
Attributes mixed in z	Intermediate space w is for **styles**; attributes vary **more independently**

L42: StyleGAN 2

Learning focus

- Name the two characteristic **StyleGAN** artifacts (water-droplet blobs and phase / alignment artifacts) and where they first show up in the synthesis pyramid.
- Explain how **AdaIN** lets the generator spike a pixel so that instance-normalization statistics become a blob.
- Write the **weight modulation** and **demodulation** formulas that replace AdaIN.
- State the three architectural changes that remove progressive growing: a **fixed** generator, **skip** `toRGB` paths, and a **residual** discriminator.

Exam lines

- StyleGAN 2 exists to kill two **structural** artifacts: **water droplets** (AdaIN statistics) and **phase / alignment** (progressive growing + high-frequency upsampling).
- Droplets: replace AdaIN with $w'_{ijk} = s_i w_{ijk}$ and demodulate by the RMS of the modulated kernel plus ε.
- Phase: freeze the pyramid, let every scale skip to RGB, give the discriminator residual paths so a deep full-res critic still preserves alignment.
- Noise injection for stochastic variation is unchanged.

Concept tables

Artifact	What you see	First visible at
Water droplet	Blob-like spots on feature maps (and sometimes on the image)	From 64×64, then **stronger** through 1024×1024
Phase / alignment	Features stay locked to the **camera center** while the face turns	Same growing pyramid

Index	Meaning
—--	———
i	input channel
j	output channel
k	spatial location inside the filter
w_{ijk}	original conv weight
w'_{ijk}	modulated weight
s_i	style scale from the affine layer

L43: Practical exercise 1 — CycleGAN (advanced GAN lab)

Learning focus

- Contrast a **vanilla GAN** (one G, one D) with **CycleGAN** (two generators, two discriminators, three loss families).
- Rebuild the **PyTorch** lab: STL-10 **horse ↔ deer**, unpaired pairing, encoder–decoder G, conv D, Adam, MSE + L1.
- Read the printed metrics (parameter counts, per-epoch losses, the six-panel grid) and the reason the checkpoints are saved.

Exam lines

- CycleGAN = G_{AB}, G_{BA}, D_A, D_B + adversarial MSE + cycle L1 + identity L1.
- This notebook: PyTorch, STL-10 class **6 horses** / class **4 deer**, 400 each, 96×96, $[-1, 1]$, batch 4, 20 epochs, Adam.
- Encoder–decoder G (ReLU, tanh out); leaky-ReLU conv D; shuffle to break pairing memory.
- Save both translation directions; D_B ended slightly stronger than D_A in the demo.

Concept tables

Model	Input	Output
G_{AB}	real horse (A)	fake deer
G_{BA}	real deer (B)	fake horse
D_A	horse-shaped images	real horse vs fake horse
D_B	deer-shaped images	real deer vs fake deer

Model	Input	Output
Loss	Role	
——	——	
Adversarial (MSE in this lab)	G fools D; D separates real/fake	
Cycle-consistency (L1)	horse → fake deer → back to horse (and the other way) should reconstruct	
Identity (L1)	$G_{AB}(\text{deer}) \approx \text{deer}$, $G_{BA}(\text{horse}) \approx \text{horse}$ — stay close when the input is already in the target domain	

Contents

Volume 05 — Diffusion (L44–L53) **1**
L44: Introduction to diffusion models 1
L45: Mathematics of diffusion models 2
L46: DDPM — forward process 3
L47: DDPM — reverse process 4
L48: Hands-on — forward diffusion process 5
L49: U-Net for denoising 5
L50: Classifier-guided diffusion model 6
L51: Classifier-free diffusion models 7
L52: Stable Diffusion model 8
L53: Hands-on reverse diffusion process 9

Volume 05 — Diffusion (L44–L53)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L44: Introduction to diffusion models

Learning focus

- Place diffusion after autoencoders, VAEs, and GANs on the course timeline, including the **stability vs. quality** gap it was meant to close.
- Read a **probability landscape**: high-p peaks = realistic images, low-p valleys = noise, and why a model needs a **compass** (local next-step choices).
- Contrast a **fixed forward** noising chain with a **learned reverse** denoising chain, and sketch training vs. inference.
- State the **Markov** property used for both chains (snakes-and-ladders example).

Exam lines

- Diffusion = many easy Markov steps instead of one intractable map from noise to data.
- Forward is **fixed** Gaussian corruption; reverse is the **learned** denoiser.
- High probability ↔ structure; low probability ↔ noise; generation is climbing with a local rule $p_t(x)$.
- The net is trained to predict **noise**, then used at inference to denoise $x_T \sim \mathcal{N}(0, I)$ down to an image.

Concept tables

Year (as taught)	Model	What it did	Limitation named here
1987	Autoencoder	Compress to a latent, **reconstruct** the input (zip / unzip a folder)	**Cannot** emit realistic **new** images; used for denoising, anomaly spotting, feature extraction, scratch restoration
2013	VAE	Latent is a **Gaussian** (μ, σ), not a single point	**Can** make variants (ship with different sky / water / hull color) but images are often **blurry**, not high-res
2015	GAN	High-quality fakes from **noise**; FaceApp (2017) and This Person Does Not Exist as GAN apps	**Mode collapse** (train on dogs/cats/horses/birds, emit only dogs) and **unstable** training
2015	Diffusion	Physics-style **diffusion** of probability	Designed to be **as stable as VAEs** and **as sharp as GANs**

Idea	Why it fails
——	———— —-
Map **every image in the world** as a peak	Impossible
Memorize **known** rooms of “your house”	No help in an **un-known house** (new images)

L45: Mathematics of diffusion models

Learning focus

- Write the **forward** kernel $q(x_t \mid x_{t-1})$ and the **reverse** kernel $p_\theta(x_{t-1} \mid x_t)$ as the lecture set them up.
- Explain the **noise variance schedule** β_t and what goes wrong if destruction is too fast or too slow.
- Recall the **Gaussian** PDF, the **standard** Gaussian $\mathcal{N}(0, 1)$, and why images need a **multivariate** Gaussian (covariance, not a scalar σ^2).
- Compute a 2×2 **covariance matrix** on the height–weight table (population n, not $n - 1$).

Exam lines

- q noising is predefined Gaussian; p_θ denoising is the trainable Gaussian.
- β_t is a **schedule**, not a constant, so x_T is fully noise without making reverse learning impossible.
- Standard Gaussian = $\mathcal{N}(0, 1)$; pixels need $\mathcal{N}(\mu, \Sigma)$.
- Σ's diagonal is variance, off-diagonal is relationship; the $n = 4$ height–weight example is the calculation you are expected to reproduce.

Concept tables

Symbol	Meaning
x_0	original data (clean image / signal)
x_t	noisy version at step t
T	total diffusion steps (can be **up to ~1000**, application-dependent)
$t \in \{1, \dots, T\}$	current step
–	Forward ———
Direction	clean → noise
Learning	none
Typical letter	q

L46: DDPM — forward process

Learning focus

- Identify **DDPM** as the 2020 Ho et al. (UC Berkeley) formulation of diffusion.
- Write the Markov forward kernel $q(x_t \mid x_{t-1})$ with mean $\sqrt{1-\beta_t}\, x_{t-1}$ and covariance $\beta_t I$.
- Show why the previous image must be **scaled** (unscaled variance **blows up**).
- Expand the recursion to the **closed-form** $q(x_t \mid x_0)$ and the **reparameterization** that samples any t in one shot.

Exam lines

- DDPM forward is a **fixed** Markov Gaussian: $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I)$.
- $\alpha_t = 1 - \beta_t$; samples $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t$.
- Skipping the $\sqrt{\alpha_t}$ scale makes variance **accumulate** until the law blows up.
- $\bar{\alpha}_t = \prod_s \alpha_s$ yields $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\, x_0,\, (1-\bar{\alpha}_t) I)$ and the one-shot $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$.

Concept tables

Forward	Reverse
Clean image → almost pure noise, step by step	Almost pure noise → clean image
No learning; predefined Gaussian	Learning lives here (next lecture)

Symbol	Meaning
x_t	image at step t
x_{t-1}	image at step $t-1$
β_t	how much noise is added at t (magnitude)
I	identity matrix

L47: DDPM — reverse process

Learning focus

- Write the reverse **Markov** joint $p_\theta(x_{0:T})$ from a Gaussian **prior** x_T.
- State the **forward posterior** $q(x_{t-1} \mid x_t, x_0)$ (mean $\tilde{\mu}_t$ depends on x_0 — unusable at generation).
- Eliminate x_0 using the forward reparameterization, replace ε by ε_θ, and sample x_{t-1}.
- Quote the **MSE noise-prediction** loss used to train DDPM.

Exam lines

- Reverse joint = prior $x_T \sim \mathcal{N}(0, I)$ times Markov conditionals $p_\theta(x_{t-1} \mid x_t)$.
- $q(x_{t-1} \mid x_t, x_0)$ is Gaussian, but $\tilde{\mu}_t$ needs the **unknown** x_0 at sampling time.
- $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t))$.
- $x_{t-1} = \mu_\theta + \sigma_t z$; train ε_θ with MSE vs true ε.

Concept tables

Factor	Meaning
$p_\theta(x_{0:T})$	joint of the **entire reverse trajectory**
$p(x_T)$	**prior** at the start of generation
$\prod p_\theta(x_{t-1} \mid x_t)$	all reverse conditionals
θ	trainable weights

Term	Lecture meaning
Prior	what you assume **before** extra evidence
Posterior	the law **after** observing more information

L48: Hands-on — forward diffusion process

Learning focus

- Implement the DDPM **forward** kernel, α_t, $\bar{\alpha}_t$, and **closed-form** x_t from x_0.
- Compare **linear** vs **cosine** β-schedules ($T = 1000$) on remaining signal and **SNR**.
- Show that **iterative Markov** noising and **one-shot** $q(x_t \mid x_0)$ match in **mean and std** (Monte Carlo).
- Train a small **time-conditioned CNN** on MNIST to predict ε, then reconstruct by **subtracting** predicted noise — and list what a full DDPM still needs.

Exam lines

- Code $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$ and the Markov one-step kernel; they match in Monte Carlo.
- $T = 1000$; cosine β beats linear on uniform signal decay and final SNR.
- Astronaut viz: linear is "done" by ~500; cosine fades evenly.
- MNIST CNN (~124k params, 12k/2k split) can drop L_{simple} and roughly undo noise in one subtract — that is **not** yet a DDPM sampler.

Concept tables

Process	Path	Who learns
Forward	$x_0 \to x_1 \to \cdots \to x_T$	nobody — add small Gaussians until noise
Reverse	$x_T \sim \mathcal{N}(0, I)$ toward x_0	**neural net**

Schedule	Shape on the plot	Remaining signal $\bar{\alpha}_t$
Linear	essentially **flat** $\beta_t \approx 1/1000$	signal decays **slowly at first**, then **drops**
Cosine	β_t **small at the start**, then **rises**	**more uniform** decline of original signal

---

L49: U-Net for denoising

Learning focus

- Place **U-Net** in DDPM: input (x_t, t) → predicted noise $\varepsilon_\theta(x_t, t)$.
- Contrast **Ronneberger** segmentation U-Net with the DDPM stack: **time embeddings**, **residual blocks**, **self-attention**.
- Compute a **sinusoidal** time embedding (the $T = 100$, $d = 8$ numerical example).
- Walk **encoder / bottleneck / decoder**, the residual-block algebra (GroupNorm, **SiLU**, time projection), and **QKV** attention.

Exam lines

- U-Net is the DDPM denoiser: global structure + skips for fine detail.
- TE(t) uses sin / cos of $t/10000^{2i/d}$; the $t = 100$, $d = 8$ vector is the exam-style drill.
- Residual path: GroupNorm → SiLU → conv → **add** W_tTE(t) → GroupNorm → SiLU → conv → $H + F(H)$.
- Attention: Q, K, V projections, softmax$(QK^\top/\sqrt{d})$, then AV, so far-apart structure (two eyes) can interact.

Concept tables

Part	Job
Encoder (contracting)	downsample; spatial size shrinks; keep important information
Bottleneck	most **compressed** summary
Decoder (expanding)	upsample / reconstruct
Skips	**crop and concatenate** encoder features onto the matching decoder level so fine detail survives

Example t	Typical look (as taught)
50	slightly noisy
500	moderately noisy
900	heavily noised

L50: Classifier-guided diffusion model

Learning focus

- Explain why an unconditional DDPM cannot choose a class (dog vs cat vs flower).
- Describe the **artist + critic** pair: a denoising U-Net plus a **noisy-image classifier**.
- Reconstruct the Bayes / score derivation that turns a classifier gradient into a **guided noise** $\hat{\varepsilon}$.
- State the **guidance scale** s trade-off (class precision vs diversity) and the sampling replacement rule.
- List the pipeline limitations that motivate classifier-free guidance in the next lecture.

Exam lines

- Unconditional DDPM generates realistic images with **no class knob**.
- Classifier guidance adds a noisy-image critic; $\nabla_{x_t} \log p_\phi(y \mid x_t)$ steers denoising toward y.
- Bayes splits the conditional score into a **realism** term (U-Net ε_θ) and a **class** term (classifier).
- Guided noise $\hat{\varepsilon}$ replaces ε_θ in the reverse update; s trades **precision vs diversity**.

- Cost: a second network, noisy-label training, per-step gradients, and adversarial artifacts. Next lecture: **classifier-free** guidance.

Concept tables

Role	What it is	What it does
Artist	Ordinary **unconditional** DDPM / U-Net	Denoises a noisy image toward something realistic. It does **not** know which class to emit.
Critic	A **classifier**, itself a pretrained network	Looks at the noisy image x_t and predicts $p_\phi(y \mid x_t)$ over classes (cats, dogs, trees, houses, ...).
s	Class recognizability	Sample diversity
—–	————————	———————
Small	Weak conditioning; images may not look like the target class	Higher diversity
Large	Strong, precise class (very "dog-like")	Diversity **drops**

L51: Classifier-free diffusion models

Learning focus

- Restate why classifier-guided diffusion is heavy (extra noisy classifier, no off-the-shelf ResNet).
- Explain **classifier-free guidance (CFG)**: one U-Net trained for **both** conditional and unconditional denoising.
- Write the CFG noise combination with guidance scale s.
- Contrast sampling cost: U-Net **+ classifier** vs **two U-Net passes**.

Exam lines

- CFG exists to **delete the noisy classifier** while keeping class control.
- One U-Net, **joint** training: keep y or replace it by null ϕ with fixed dropout probability.
- The old classifier gradient is rewritten as **conditional minus unconditional** diffusion scores.
- $\hat{\varepsilon} = (1+s)\varepsilon_\theta(x_t, t, y) - s\varepsilon_\theta(x_t, t, \phi)$; larger s (paper demo $s = 3$) sharpens class at some cost (saturation / diversity).
- Sampling trades classifier cost for **two U-Net evaluations**. Next lecture: **Stable Diffusion**.

Concept tables

Problem	Why it hurts
Extra classifier	Must be trained **on noisy images** x_t to supply $\nabla_{x_t} \log p(y \mid x_t)$
No ImageNet ResNet	Off-the-shelf nets saw **clean** pixels, not x_t
Two trainings	Unconditional DDPM **and** the classifier → slower, more complex pipeline
	Classifier-guided (Lec 50)
–	——————————-
Separate classifier	Yes
Noisy-classifier training	Yes
Where the guidance signal comes from	Classifier gradient $\nabla_{x_t} \log p_\phi(y \mid x_t)$
Sampling-time cost	U-Net **plus** classifier, both run while generating

---

L52: Stable Diffusion model

Learning focus

- Motivate **latent** diffusion: why pixel-space DDPM on $512 \times 512 \times 3$ is too expensive.
- Separate **latent diffusion (LDM)** (the algorithm) from **Stable Diffusion** (text-to-image implementation).
- Trace the frozen autoencoder → noisy latent → U-Net → decoder pipeline and the latent-space MSE objective.
- Write **cross-attention** with Q from U-Net maps and K, V from a **CLIP** text encoder.
- Choose **concatenation vs cross-attention** from whether the condition has spatial structure.

Exam lines

- LDM moves DDPM from pixels into a **frozen autoencoder's latent** ($64 \times 64 \times 4$ vs $512 \times 512 \times 3$).
- Stable Diffusion **is** that algorithm plus **CLIP text embeddings** and **cross-attention** into the U-Net.
- Loss is DDPM's noise MSE on z_t, not x_t.
- Q from U-Net maps, K and V from $\tau_\theta(y)$; concat is for **spatial** conditions.
- Next in the playlist is the **reverse-diffusion lab**; the instructor also flags the coming shift to **NLP / transformers / LLMs**.

Concept tables

Name	What it is
Latent diffusion model (LDM)	The **algorithm** in that paper

Name	What it is
Stable Diffusion	A **practical** LDM implementation for **text-to-image**, with extra **conditioning**
Symbol	Meaning
——–	———
ε	Gaussian noise added in the **forward** process
z_t	**Noisy latent** at time t
$\varepsilon_\theta(z_t, t)$	U-Net prediction — depends on z_t **and** t, **not** on pixel x_t

---

L53: Hands-on reverse diffusion process

Learning focus

- Implement reverse DDPM sampling from $\mathcal{N}(0, I)$ with a **small class-conditional U-Net** on **MNIST**.
- Train with L_{simple} (MSE on Gaussian noise) and **null-class dropout** for classifier-free guidance.
- Compare **guidance scales** and a tiny **quality classifier** on generated digits.
- Contrast diffusion vs **GAN** vs **VAE** using the lecture's table (tendencies, not universal laws).

Exam lines

- Reverse diffusion repeatedly maps noise toward data; a U-Net mixes global structure with skip-connection detail.
- Train by predicting known Gaussian noise with **MSE**; condition on t (noise level) and optionally y.
- Null-class dropout + $\varepsilon_{\text{uncond}} + s(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}})$ is **classifier-free** guidance. Moderate s helps class adherence; excessive s hurts diversity / adds artifacts.
- This classroom model **demonstrates the mechanism**; scale data, T, channels, and epochs to get convincing digits. That closes the diffusion hands-on pair (forward + reverse).

Concept tables

Setting	Value in the notebook
Time steps T	**100** (`fast_mode=True`; set `False` to increase T)
β schedule	**Linear**, β from **0** to **0.02**
α_t	$1 - \beta_t$
Also computed	$\bar{\alpha}_t$, posterior variance
Split	Size
——-	——
Train	**12,000**
Validation	**2,000**

Contents

Volume 06 — NLP, Transformers, LLMs (L54–L63) **1**
L54: Foundations of natural language processing 1
L55: Sequential modeling with RNNs and LSTM 2
L56: Evolution from LSTMs to transformers 3
L57: Transformer architecture — encoder 4
L58: Practical exercise 1 — RNN vs LSTM sentiment classification 5
L59: Transformer architecture — decoder 6
L60: BERT 7
L61: GPT — decoder-only large language model 8
L62: Prompt engineering basics 9
L63: Hands-on on LLM 10

Volume 06 — NLP, Transformers, LLMs (L54–L63)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L54: Foundations of natural language processing

Learning focus

- Define NLP as mapping human language to something a model can compute, then to an action.
- Walk the pipeline: **raw text → preprocessing → numerical representation → model**.
- Tokenize at word / sentence / **subword** / character granularity; apply stop-word removal and **lemmatization**.
- Contrast **Word2Vec** (local windows, king–queen arithmetic) with **GloVe** (co-occurrence matrix).
- List NLP challenges that motivate **sequential** models in Lec 55 (especially **long-range dependency**).

Exam lines

- NLP turns language into **clean tokens**, then **vectors**, then a task output (spam, sentiment, translation, QA, …).
- Preprocessing: lowercase, strip punct, **tokenize**, drop stop words, **lemmatize**.
- Word2Vec: local windows + vector arithmetic (king − man + woman ≈ queen).
- GloVe: **co-occurrence** counts → dense embeddings.
- Discrete, drifting, ambiguous language plus **long-range** binding still break bag-of-features thinking — hence sequence models next. Contextual BERT/GPT embeddings wait for the transformer week.

Concept tables

Input	Output	Action / task
“Is this email spam?”	Yes	**Spam detection**
“The movie was amazing”	Positive	**Sentiment analysis**
“Hello”	“Hola”	**Machine translation**
“What is generative AI?”	A definition paragraph	**Question answering**
Stage	What happens	Example
——-	——————-	———
Raw text	User text, or speech converted to text	Full sentence + punctuation
Text preprocessing	Clean, normalize, drop extras	Content words such as **absolutely / love / movie**
Text representation	Words → **numerical vectors** (models do not read strings)	Embedding per token
Model	Learns patterns from those vectors	Sentiment model
Output	Task prediction	**Positive** sentiment

L55: Sequential modeling with RNNs and LSTM

Learning focus

- Motivate sequence models from **word order** (“dog bites man” vs “man bites dog”).
- Distinguish **short-term** vs **long-term** temporal dependence.
- Write the RNN **hidden-state** and **output** updates; map **one-to-one / one-to-many / many-to-one / many-to-many**.
- Explain **vanishing / exploding** gradients and **BPTT** as why vanilla RNNs fail on long gaps.
- Walk LSTM **forget, input, output** gates and the **cell state** highway.

Exam lines

- Order is meaning: same bag of words, different sequence, different sentence.
- Feedforward nets have **no** sequential memory; RNNs store it in $h_t = \tanh(Vh_{t-1} + Ux_t + b_h)$.
- RNN patterns match captioning, sentiment, translation / tagging.
- Long gaps + BPTT → **vanishing / exploding** gradients.
- LSTM adds **cell state** c_t and three gates so memory can survive many steps.
- **Still not enough** when dependencies are very long → **transformers / attention** in Lec 56.

Concept tables

Sentence	Meaning
The dog bites man	Dog is the biter
Man bites dog	Same words, **different** event
Matrix	Connects
——–	———-
U	Input x_t → hidden h_t
V	**Recurrent:** previous hidden h_{t-1} → current h_t
W	Hidden h_t → output $\hat{y}_t$

L56: Evolution from LSTMs to transformers

Learning focus

- Place RNN → LSTM → **attention (2014)** → **transformers (2017)** on the lecture's timeline.
- Explain why recurrence is expensive for long documents ($O(n)$ sequential steps, memory of every h_t).
- Define **encoder–decoder attention** (context $c_t = \sum_j \alpha_{tj} h_j$) and **self-attention** (Q, K, V).
- Motivate **multi-head** attention and **causal masks** ($-\infty$ → softmax 0).
- Contrast RNN/LSTM vs transformers: sequential vs **parallel**, hidden state vs **direct** token links.

Exam lines

- Recurrence forces $O(n)$ hops and a stack of hidden states; long documents (paragraph-scale) become expensive and forgetful.
- Attention **reweights** encoder states (or tokens) so it can lock onto animal, not the whole sentence equally.
- Self-attention: Q, K, V from the **same** sequence; multi-head = several relations at once; **masks** implement causal decoding.
- Transformers (2017) combine attention with **parallel** reads of the whole sequence — the reason they replace LSTMs as the default sequence backbone. Next lecture: **transformer architecture** in detail.

Concept tables

When	What
~2013	**RNNs** in wide NLP use
1997 / adopted ~2014	**LSTMs** (Hochreiter & Schmidhuber 1997; popular after 2014)
2014	**Attention** (the ingredient transformers will rest on)
2017	**Transformers** — NLP landscape changes; leads to **BERT, GPT, LLaMA**, ...

When	What
Vector	Informal job (lecture)
——–	———————-
Query	What am I looking for?
Key	What information do I contain?
Value	What should I pass forward?

L57: Transformer architecture — encoder

Learning focus

- State why RNNs, LSTMs, and GRUs motivated the 2017 transformer: sequential compute, hard parallelization, and remaining recurrence after attention.
- Describe the encoder as **six identical stacked layers**, each with **multi-head self-attention** and a **position-wise feed-forward network**.
- Explain why **sinusoidal positional encodings** are added to token embeddings when every token is processed at once.
- Compute a $d_{\text{model}} = 4$ positional encoding for the token sat in "the cat sat on the mat."
- Write residual + layer-norm and the two-linear ReLU FFN used in the original paper.

Exam lines

- Seq2seq recurrence made training slow, blocked parallelization, and still struggled with **very** long range; Attention Is All You Need (2017) dropped recurrence and convolutions.
- The encoder is **six identical layers** (different weights): multi-head self-attention + position-wise FFN, each wrapped in residual Add & Norm.
- Encoder **understands** all input tokens at once; decoder **generates** one token at a time from past outputs **and** encoder context (English→German).
- Simultaneous encoding loses order, so **sinusoidal PE** is added to embeddings; even dims sine, odd dims cosine.
- Residual $x+\text{Sublayer}(x)$ keeps information from being wiped; layer-norm stabilizes training; FFN is $512 \to 2048 \to \text{ReLU} \to 512$.

Concept tables

Limitation	What happens
Sequential computation	Word 1, then 2, then 3, … Training is slow.
Hard to parallelize	Later tokens wait on earlier hidden states; many operations cannot run together.
Very long-range dependency	LSTMs help, but words many sentences apart are still a problem.
Block	Job

Limitation	What happens
——-	—–
Encoder	**Understand** the English input. Process **all input tokens simultaneously**. Emit a **contextual representation** of every token.
Decoder	**Generate** the German translation. At **inference**, it does **not** emit all words at once: **one word at a time**.

L58: Practical exercise 1 — RNN vs LSTM sentiment classification

Learning focus

- Implement **binary sentiment classification** on **IMDb** with a **SimpleRNN** and an **LSTM** in TensorFlow / Keras.
- Pad / truncate reviews to a fixed length with **post-padding** and **post-truncation**.
- Compare validation accuracy/loss, test accuracy, and confusion matrices.
- Inspect **LSTM hidden-state** heatmaps (tanh activations) and decode two test reviews.

Exam lines

- IMDb 25k/25k, vocab **10k**, length **200**, **post-pad**; Embedding **128**, recurrent **64**, sigmoid + **Adam** + **BCE**, 10 epochs, batch 64, 20% validation.
- SimpleRNN stuck near **50%**; LSTM near **85% val / 82% test** because of **forget / input / output** gates and a **cell state**.
- Hidden-state heatmaps show tanh activity on real tokens, then **constant** rows on **padding**.
- This exercise is **sentiment RNNs**, not a transformer; transformer training/inference is Lec 63.

Concept tables

Split	Reviews
Train	25,000
Test	25,000
Total	50,000
Parameter	Value
———	——:
`vocab_size` / `num_words`	**10,000**
`maxlen`	**200**

L59: Transformer architecture — decoder

Learning focus

- List the decoder's **three** sub-layers and why the encoder only needed two.
- Explain **masked** multi-head attention vs **encoder–decoder** attention (Q from decoder, K, V from encoder).
- Reconstruct **BPE** (frequency) vs **WordPiece** (likelihood) as the pre-processing step before embeddings.
- Recall the **WMT 2014** En–De / En–Fr training setup and transformer **advantages vs quadratic cost**.

Exam lines

- Decoder layer = **masked self-attention** + **encoder–decoder attention** (Q decoder, K, V encoder) + **FFN**, each with residual Add & Norm; **six** layers in the paper.
- Masking stops future-target leakage; cross-attention injects source context; softmax over the vocab emits the next word.
- Extra sub-layer exists because generation has **two** look-ups (past outputs **and** encoder).
- BPE (frequency) and WordPiece (likelihood) shrink vocab and handle unknowns (cyber+security).
- Trained on WMT 2014 En–De (~4.5M, BPE) and En–Fr (~36M, WordPiece); pay $O(n^2)$ attention cost.

Concept tables

Side	Job	Attention it needs
Encoder	Only **understand** the input	Self-attention over the source + FFN (**two** sub-layers)
Decoder	**Two** jobs	(1) Look at **already generated** words → **masked** self-attention. (2) Attend to **encoder** context → **cross-attention**. Then FFN.

Model (as stated)	Heads
————————-	——:
Original transformer	**8**
Transformer **big**	**16**

L60: BERT

Learning focus

- Contrast BERT with **static** embeddings (Word2Vec / GloVe) and **left-to-right** LMs using the lecture's bank and movie was — examples.
- Explain why BERT is **encoder-only**: **understand** language, not generate long text.
- Describe the two pre-training tasks **MLM** and **NSP**, including the **15% / 80–10–10** masking recipe.
- Trace input construction: **token + segment + position** embeddings, `[CLS]`, `[SEP]`.
- Distinguish **self-supervised pre-training** from **supervised fine-tuning**, and name the BERT **variants** listed in class.

Exam lines

- Static bank vectors and left-to-right LMs miss context; BERT is a **bidirectional encoder** LM (Google / Devlin et al.).
- Encoder-only because target tasks need **labels and short answers**, not free generation.
- Pre-train **MLM** (15% selected; 80% `[MASK]`, 10% random, 10% unchanged) and **NSP** (50/50 IsNext) on BooksCorpus + Wikipedia with WordPiece.
- Input = token + position + segment; `[CLS]` for classification; fine-tune with a small head on labeled data.
- Variants: RoBERTa, ALBERT, DistilBERT, ELECTRA — still BERT-family encoders, not GPT.

Concept tables

Task	Input (lecture)	Output
Sentiment	I absolutely love this movie.	**positive** (one label, not a sentence)
NER	Dr. Joan visited Ames Hospital on August 1st, 2025.	person / organization / date
QA	Paragraph: Paris is the capital of France. Q: What is the capital of France?	**Paris**
Text classification	(same idea)	class label

Stage	School analogy	Data
Pre-training	KG through class 12: many subjects	**Massive unlabeled** corpus
Fine-tuning	Medical school: specialize	**Smaller labeled** set

L61: GPT — decoder-only large language model

Learning focus

- Contrast **BERT (encoder, understanding, MLM)** with **GPT (decoder, generation, next-token)** as taught in this pair of lectures.
- Explain **causal / masked self-attention** and why a **bidirectional encoder cannot** generate without **information leakage**.
- Walk the **autoregressive** loop on The capital of France is … Eiffel Tower.
- Describe GPT's **pre-train (unlabeled next-word) → supervised fine-tune** framework and the **decoder-only** block (no encoder–decoder attention).
- List GPT-1 vs GPT-2 scale numbers and the **limitations** named here (including **hallucination**). Prompt recipes are Lec 62 — not this video.

Exam lines

- GPT-1 (OpenAI / Radford, 2018) is **decoder-only**; BERT is **encoder-only**. GPT predicts the **next** token; BERT predicted **masks**.
- Causal masking enables generation; bidirectional encoder attention would **leak** future tokens.
- Generation is **autoregressive**: append Paris, then `.`, then It is known for the Eiffel Tower.
- Pre-train unlabeled next-word (12 decoder blocks, no cross-attention) → fine-tune with small linear heads (classification, entailment, similarity, MCQ).
- GPT-1 → GPT-2: 117M→1.5B parameters, 5→40 GB, 512→1024 context, 40k→50k BPE. Failures named here: hallucination, bias, cost, context, prompts, cutoff, safety — not RAG/ethics weeks.

Concept tables

Year (lecture)	Model
2018	**GPT-1** (same year as BERT)
2019	**GPT-2**
2020	**GPT-3**
2022	**GPT-3.5**
2023	**GPT-4**
2024	**GPT-4o**
2025	**GPT-5**, and the lecture's latest mention **GPT-5.6**
	BERT
–	———-
Aim	Language **understanding**
Stack	**Encoder-only** transformer
Pre-train prediction	**Masked** token (both sides): the boy is **[MASK]** football → playing

L62: Prompt engineering basics

Learning focus

- Define a **prompt** as the input that **guides** an LLM, and **prompt engineering** as designing that input so the answer matches what you asked for.
- List what a prompt can contain: **instructions, questions, context, constraints, examples**.
- Name the six techniques taught: **zero-shot, one-shot, few-shot, chain-of-thought, role, structured-output**.
- Contrast a **vague** vs **specific** prompt using the explain machine learning pair.

Exam lines

- A prompt is the **guiding input**; engineering it means instructions + context + constraints + examples so the LLM matches intent.
- Good prompts are clear, contextual, specific, audience- and format-aware; avoid vagueness, missing context (hallucinated background), and unrelated question piles.
- Six named techniques: **zero-shot** (no demos), **one-shot**, **few-shot** (2–5), **chain-of-thought** (step-by-step $5/$2 shop), **role** (physics professor / $F = ma$ cart), **structured output** (JSON / bullets / code).
- Explain machine learning vs 150 words, first-years, one example is the running quality contrast.
- RAG, ethics, and LoRA are **not** in this transcript.

Concept tables

Ingredient	Lecture example
Instructions	Summarize this article in **five bullet points**
Context	The article itself, pasted in
Questions	Follow-ups (e.g. weather in Bangalore over five months **from that summary**)
Constraints	Avoid rainy season; only summer/winter/…
Examples	Demonstrations of the desired pattern
Role of the prompt	What it does
———————–	————–
Define the task	Summarization, translation, explanation, classification, QA, **code generation**, …
Provide context	Background so the situation is understood
Specify the audience	Explain AI to a **layperson** vs **PhD / research** scholars → **depth** changes
Control format	Bullets, **table**, code, essay; “three bullet points” vs a long paragraph
Reduce ambiguity	Task + context + audience + format + extra specifics → fewer misreads

L63: Hands-on on LLM

Learning focus

- Implement **scaled dot-product attention** with a **causal (triangular) mask** and **next-token** training (input shifted by one).
- Train a **tiny decoder-only** transformer from scratch on a **character** corpus (paragraph × 80).
- Decode with **greedy**, **top-k**, and **nucleus (top-p)** sampling; see **temperature** and **seed**.
- Compare **character** vs **BPE / WordPiece** on Hugging Face tokenizers; run **DistilGPT-2** as a pretrained **completion** model (not instruction-tuned chat).
- Treat **prompts** as incomplete context. This notebook does **not** implement RAG.

Exam lines

- Causal LMs learn **next-token** prediction; **causal masks** block future tokens in train and decode.
- Tokenization is **model-specific**: use the **same** tokenizer at train and inference.
- **Greedy** is deterministic (seed-invariant); **top-k** and **nucleus** add **controlled** diversity.
- Prompt wording **shifts** the continuation distribution; a prompt is **unfinished context**, not a separate RAG index.
- Tiny char-LM (block 64, Adam 3×10^{-4}, loss ~3.6→0.16) plus DistilGPT-2 decoding is the whole lab — **no RAG, no ethics module**.

Concept tables

Hyperparameter	Value	Lecture name
Block size	**64**	context window (64 **characters** per sample)
Batch size	32 (from printed shapes)	independent sequences per step
Split	**90%** train / **10%** val	prefix of the token stream vs rest

Mode	Rule
——	——
Greedy	Always take the **max** probability token. **No stochasticity** — same string every time (seed / temperature do not change it).

Hyperparameter	Value	Lecture name
Top-k	Keep the k **largest** logits; sample from that set (with temperature, e.g. 1).	
Nucleus / top-p	Smallest prefix of the ranked vocab whose probabilities **sum to ≥** p (e.g. **0.9**); sample from that set.	